CLAUDE CODE SOURCE STUDY · VOL. 05

내구성, 복구와 Agent Lifecycle

append-only transcript, DAG 복원, compaction, abort, background task와 child agent가 부분 실행을 어떻게 수습하는지 분석한다

> 프로젝트의 구조를 외우는 대신, 코드가 어떤 상태와 불변조건을 지키며 결정적인 동작을 보장하는지 이해하기 위한 스터디 보고서다.

이 권에서 얻게 될 통찰

이 권을 마치면 resume를 deserialize가 아니라 journal·workspace·protocol 사이의 reconciliation으로 설명할 수 있다.

Fixed source
Claude Code v2.1.88 복원 snapshot
2ca5ddabfed5f220812ea11f029eda03b21bc4c1
2026-09-20 · Korean study edition

READING BASELINE

OBSERVED

먼저, 지금 보고 있는 시스템의 상태를 고정한다

message와 metadata는 JSONL 계열 record로 쌓이고 parentUuid chain에서 conversation을 재구성한다. session memory는 threshold에 따라 별도 restricted agent가 갱신하는 파생 파일이다.

구현에서 직접 확인되는 범위

병렬 tool result는 DAG를 만들 수 있어 별도 recovery pass가 orphan sibling/result를 복원한다. compaction, session memory, CLAUDE.md는 서로 다른 authority와 cutoff를 가지며 AgentTool은 nested query runtime을 연다.

운영을 말할 때 남겨야 할 경계

중단·compaction·background task·memory extraction은 durable truth와 UI/context projection을 다르게 만든다.

이 보고서가 추측으로 채우지 않는 것

crash consistency의 전체 보증, 원격 ingress의 production ordering, 누락 context-collapse module, 그리고 production session-memory gate와 threshold 값은 현재 snapshot만으로 확정할 수 없다. 이후의 설명은 이 공백을 유지한 채, 실제 코드가 보여 주는 구조와 보장만을 다룬다.

읽는 동안 반복할 질문

이 문장은 코드로 관찰한 사실인가, 구조에서 끌어낸 해석인가, 아니면 개선을 위한 제안인가?

SYSTEM READING MAP

OBSERVED

파일이 아니라 책임과 보장의 연결로 읽는다

이 지도는 번호를 따라 구성요소를 외우게 하지 않는다. 각 영역이 어떤 의미를 소유하고, 그 결과가 다음 영역의 판단 근거가 되며, 마지막에 어떤 외부 보장으로 닫히는지를 문장으로 연결한다.

기록·복구

Journal write `recordTranscript/entries`

append intent

"append intent"가 확인되어야 다음의 "Topology" 영역이 "chain + DAG"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

상태 소유

Topology `UUID/parentUuid`

chain + DAG

"chain + DAG"가 확인되어야 다음의 "Replay" 영역이 "consistent history"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

진입 경계

Replay `load/build/recover`

consistent history

"consistent history"가 확인되어야 다음의 "Compaction" 영역이 "lossy checkpoint"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

기록·복구

Compaction `boundary + summary`

lossy checkpoint

"lossy checkpoint"가 확인되어야 다음의 "Agents/tasks" 영역이 "child lifecycle"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

상태 소유

Agents/tasks `nested query + task state`

child lifecycle

"child lifecycle"가 확인되어야 다음의 "Settlement" 영역이 "effect truth"를 자신의 보장으로 이어 받을 수 있다. 두 구현의 호출 관계만 보지 말고 그 사이에서 전달되는 값의 정체성과 완료 조건을 함께 확인한다.

관찰·정산

Settlement `read-back/reconcile`

effect truth

이 책임은 앞선 판단을 외부에서 관찰 가능한 결과로 닫는다. 이 지점의 실패 처리가 모호하면 앞의 모든 단계가 성공했더라도 시스템 전체는 정산되지 않은 상태로 남는다.

선을 따라갈 때에는 구현 이름보다 인접한 두 책임 사이에서 전달되는 identity, state, completion을 확인한다. 그 셋 중 하나가 사라지는 지점이 구조적 결함이 나타날 가능성이 가장 높은 경계다.

01 Transcript를 journal로 읽는다

저장 파일은 화면 snapshot이 아니라 여러 projection을 다시 만드는 재료다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	책임 연결 Transcript write는 chain identity를 남긴다	알고리즘	책임 연결 Leaf-to-root conversation chain reconstruction
알고리즘	효과·위임 병렬 tool result orphan recovery	알고리즘	관찰·정산 Resume consistency telemetry
구분	기록·복구 Journal·workspace·protocol truth	사례	책임 연결 사례: Edit는 commit됐지만 result append 전에 종료

코드 독해의 시작점

`src/utils/sessionStorage.ts` → `src/types/logs.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Transcript write는 chain identity를 남긴다

새 message를 append할 때 UUID와 parent 관계가 이후 resume topology를 결정한다.

transcript and sidechain record

src/utils/sessionStorage.ts:1408-1465

```
1408  export async function recordTranscript(
1409    messages: Message[],
1410    teamInfo?: TeamInfo,
1411    startingParentUuidHint?: UUID,
1412    allMessages?: readonly Message[],
1413  ): Promise<UUID | null> {
1414    const cleanedMessages = cleanMessagesForLogging(messages, allMessages)
1415    const sessionId = getSessionId() as UUID
1416    const messageSet = await getSessionMessages(sessionId)
1417    const newMessages: typeof cleanedMessages = []
1418    let startingParentUuid: UUID | undefined = startingParentUuidHint
1419    let seenNewMessage = false
1420    for (const m of cleanedMessages) {
1421      if (messageSet.has(m.uuid as UUID)) {
1422        // Only track skipped messages that form a prefix. After compaction,
1423        // messagesToKeep appear AFTER new CB/summary, so this skips them.
1424        if (!seenNewMessage && isChainParticipant(m)) {
1425          startingParentUuid = m.uuid as UUID
1426        }
1427      } else {
1428        newMessages.push(m)
1429        seenNewMessage = true
1430      }
1431    }
1432    if (newMessages.length > 0) {
… 28 lines omitted; use the source link for the complete range …
1461    )
1462  }
1463
1464  export async function recordQueueOperation(queueOp: QueueOperationMessage) {
1465    await getProject().insertQueueOperation(queueOp)
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	main chain과 sidechain write가 나뉜다.
그 표현이 만든 조건	cleanMessagesForLogging projection을 거친다.
그 표현이 만든 조건	startingParentUuid를 반환/전달한다.
그 표현이 만든 조건	queue operation도 별도 record다.

언어와 런타임은 여기까지 보장한다

append-only journal은 in-place state보다 history를 보존하지만 올바른 replay algorithm이 필요하다.

프로젝트가 이 메커니즘에 맡긴 책임

화면에 보이는 순서와 disk parent topology가 동일하다고 가정하면 안 된다.

경계를 잘못 읽을 때 생기는 위험

write 중단과 중복 UUID를 replay가 처리해야 한다.

이 설명을 확인할 코드 src/utils/sessionStorage.ts:1408-1465

ALGORITHM A25 · 독자의 질문

Leaf-to-root conversation chain reconstruction은 어떤 시스템 문제를 닫는가

transcript 파일은 append 순서대로 읽는 단순 log가 아니라 parentUuid로 branch가 갈리는 graph다. 사용자가 선택한 leaf에서 어떤 대화가 실제 ancestry인지 복원해야 resume context가 정해진다.

코드가 필요해지는 이유

한 파일의 여러 branch/metadata 중 선택된 conversation lineage를 복원한다.

OBSERVED buildConversationChain이 UUID map, 선택 leaf, seen set, 역방향 walk 결과를 소유한다.

INFERENCE storage는 모든 record를 보존하고 chain builder가 한 resume view의 canonical lineage를 선택한다.

이 절에서 답할 결정 질문

leaf에서 parent를 거슬러 올라갈 때 missing edge와 cycle을 어떻게 다루고 sibling을 왜 자동 포함하지 않는가?

이 설명을 확인할 코드 src/utils/sessionStorage.ts:2069-2117

ALGORITHM A25 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/sessionStorage.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
linked lineage와 cycle detection

각 node가 parent 하나를 가리키면 leaf-to-root walk로 한 계보를 복원할 수 있다. seen set은 malformed cycle에서 무한 loop를 막지만 누락된 branch를 자동 복구하지는 않는다.

> **실제 코드로 돌아가면.** 2069–2117행에서 parent lookup 실패와 seen 처리, 마지막 reverse 순서를 본다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`linked lineage · cycle detection`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

내구성·복구
`event replay`
중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

ALGORITHM A25 · 최소 예제

append 순서와 선택된 lineage는 같지 않다

한 parent에서 두 branch가 갈라진 transcript를 축소한다.

설명을 위한 최소 실행 예제

```
root <- a <- leafA
        \ b <- leafB
selected = leafB
chain = [root, b, leafB] // a, leafA는 파일에 있어도 제외
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
map 구성	모든 UUID record가 존재	저장된 집합과 선택 context는 다르다.
leafB walk	parent를 따라 b→root	하나의 ancestry만 얻는다.
reverse	root→b→leafB	provider에 시간 순 context를 제공한다.

예제를 실제 구현에 다시 연결한다

실제 함수가 file order가 아니라 parent relation을 authority로 삼는 이유다.

ALGORITHM A25 · 실제 코드

OBSERVED

Leaf-to-root conversation chain reconstruction의 결정점은 어디에 있는가

resume은 최신 leaf에서 parentUuid를 따라가며 root-to-leaf history를 만든다.

입력 표현. UUID→TranscriptMessage map, 그리고 선택된 leaf

보존되는 작업 상태. transcript array, seen UUID set, 그리고 current message

buildConversationChain

src/utils/sessionStorage.ts:2069-2117

```
2069  export function buildConversationChain(
2070    messages: Map<UUID, TranscriptMessage>,
2071    leafMessage: TranscriptMessage,
2072  ): TranscriptMessage[] {
2073    const transcript: TranscriptMessage[] = []
2074    const seen = new Set<UUID>()
2075    let currentMsg: TranscriptMessage | undefined = leafMessage
2076    while (currentMsg) {
2077      if (seen.has(currentMsg.uuid)) {
2078        logError(
2079          new Error(
2080            `Cycle detected in parentUuid chain at message ${currentMsg.uuid}. Returning partial
transcript.`,
2081          ),
2082        )
… 32 lines omitted; use the source link for the complete range …
2115    * Read-side fix: the write topology is already on disk for old transcripts;
2116    * this recovery pass handles them.
2117    */
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A25 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

UUID→TranscriptMessage map, 그리고 선택된 leaf은 처음부터 신뢰된 실행 상태가 아니다. transcript array, seen UUID set, 그리고 current message가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** leaf를 transcript에 넣는다. 이 결과가 다음 판단 "parentUuid로 이전 message를 찾는다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** parentUuid로 이전 message를 찾는다. 이 결과가 다음 판단 "cycle이면 partial chain과 telemetry를 남기고 중단한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** cycle이면 partial chain과 telemetry를 남기고 중단한다. 이 결과가 다음 판단 "root에 도달하면 reverse한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** root에 도달하면 reverse한다. 이 결과가 다음 판단 "병렬 orphan recovery pass를 실행한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 병렬 orphan recovery pass를 실행한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

같은 UUID를 두 번 방문하지 않는다. 결과 순서는 root→leaf다. broken parent를 조용히 다른 branch로 대체하지 않는다.

ALGORITHM A25 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** 같은 UUID를 두 번 방문하지 않는다.
- **내구성·복구의 질문으로 읽기.** 결과 순서는 root→leaf다.
- **상태·계산 모델의 질문으로 읽기.** broken parent를 조용히 다른 branch로 대체하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- missing parent
- cycle
- 병렬 sibling이 single-parent walk에서 탈락

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED append record를 직접 정렬하기보다 explicit parent relation을 canonical lineage로 사용한다.

RECOMMENDATION replay result에 repair ledger와 missing edge를 함께 반환한다.

ALGORITHM A25 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

selected leaf에서 parentUuid를 따라가 seen set에 넣고 root 도달 뒤 뒤집는다.

effect가 시작되고 관찰되는 경계

복원된 chain이 다음 model context가 되므로 제외된 branch는 resume에서 보이지 않는다.

현재 설계가 선택한 trade-off와 비용

append record를 직접 정렬하기보다 explicit parent relation을 canonical lineage로 사용한다. 이 선택의 계산·운영 비용은 주 chain 길이 L에 O(L), 후속 recovery 별도.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

missing parent, cycle, 병렬 sibling은 서로 다른 repair 필요성을 가진다. 이때 함께 대입할 반례는 missing parent, cycle, 그리고 병렬 sibling이 single-parent walk에서 탈락이다. 이들을 모두 하나의 “실패”로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

selected leaf, missing edge, cycle stop, excluded sibling count를 repair ledger로 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A25 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	UUID map과 seen set으로 leaf-to-root chain을 만든다.
CONDITIONAL	single-parent link가 온전한 lineage에서는 canonical resume order를 얻는다.
UNKNOWN	저장 중 부분 write가 만든 모든 graph 손상 패턴과 복구 정책은 알 수 없다.
NEXT READ	A26 orphan recovery가 기본 chain 밖의 어떤 병렬 result만 선택적으로 삽입하는지 이어서 본다.

다른 입력과 변경안에 전이해 볼 질문

1. 선택 leaf의 parent가 없으면 root로 간주할 것인가 불완전 chain으로 표시할 것인가?
2. 두 sibling branch를 모두 합치면 사용자 대화의 causal order가 왜 달라질 수 있는가?

이 설명을 확인할 코드 src/utils/sessionStorage.ts:2069-2117

ALGORITHM A26 · 독자의 질문

병렬 tool result orphan recovery 은 어떤 시스템 문제를 닫는가

병렬 tool result는 하나의 assistant call에서 여러 sibling branch로 기록될 수 있어 single-parent walk가 일부를 놓친다. legacy data를 다시 쓰지 않고 read 시점에 필요한 sibling만 안정된 위치에 복구한다.

코드가 필요해지는 이유

병렬 tool_use가 만든 DAG를 linked-list traversal만으로 읽을 때 branch가 사라지는 문제를 고친다.

OBSERVED — recovery pass가 message.id sibling group, assistant UUID별 result, anchor별 insert list를 만든다.

INFERENCE — 기본 lineage 복원 뒤 legacy topology를 보정하는 read-side projection repair다.

이 절에서 답할 결정 질문

orphan을 어떤 group과 anchor에 연결하고, timestamp tie에서도 재생 순서를 안정적으로 만드는가?

이 설명을 확인할 코드 src/utils/sessionStorage.ts:2118-2205

ALGORITHM A26 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/utils/sessionStorage.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
DAG linearization과 stable sort

부분 순서만 있는 graph를 한 배열로 만들 때 causal edge를 지키면서 tie의 원래 순서를 보존해야 한다. 단순 timestamp sort는 clock anomaly에서 의미를 잃는다.

> **실제 코드로 돌아가면.** 2118–2205행에서 orphan 후보, anchor 선택, sort key와 insert order를 따로 본다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`DAG linearization · stable sort`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

내구성·복구
`backward-compatible repair`
중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

ALGORITHM A26 · 실제 코드

OBSERVED

병렬 tool result orphan recovery 의 결정점은 어디에 있는가

single-parent chain이 잃은 sibling assistant와 tool_result를 message.id와 parentUuid index로 되찾는다.

입력 표현. 전체 message map, 기본 chain, 그리고 seen set

보존되는 작업 상태. message.id별 sibling group, assistant UUID별 tool result, 그리고 anchor별 insert list

recoverOrphanedParallelToolResults

src/utils/sessionStorage.ts:2118-2205

```
2118  function recoverOrphanedParallelToolResults(
2119    messages: Map<UUID, TranscriptMessage>,
2120    chain: TranscriptMessage[],
2121    seen: Set<UUID>,
2122  ): TranscriptMessage[] {
2123    type ChainAssistant = Extract<TranscriptMessage, { type: 'assistant' }>
2124    const chainAssistants = chain.filter(
2125      (m): m is ChainAssistant => m.type === 'assistant',
2126    )
2127    if (chainAssistants.length === 0) return chain
2128
2129    // Anchor = last on-chain member of each sibling group. chainAssistants is
2130    // already in chain order, so later iterations overwrite → last wins.
2131    const anchorByMsgId = new Map<string, ChainAssistant>()
… 71 lines omitted; use the source link for the complete range …
2203      if (toInsert) result.push(...toInsert)
2204    }
2205    return result
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A26 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

전체 message map, 기본 chain, 그리고 seen set은 처음부터 신뢰된 실행 상태가 아니다. message.id별 sibling group, assistant UUID별 tool result, 그리고 anchor별 insert list가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** on-chain assistant를 message.id로 anchor한다. 이 결과가 다음 판단 "전체 map을 한 번 순회해 sibling/result index를 만든다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 전체 map을 한 번 순회해 sibling/result index를 만든다. 이 결과가 다음 판단 "on-chain group마다 unseen sibling과 result를 모은다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** on-chain group마다 unseen sibling과 result를 모은다. 이 결과가 다음 판단 "timestamp stable sort한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** timestamp stable sort한다. 이 결과가 다음 판단 "마지막 on-chain group member 뒤에 삽입한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 마지막 on-chain group member 뒤에 삽입한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

같은 recovered UUID를 중복 삽입하지 않는다. assistant sibling이 tool_result보다 먼저 온다. group은 API normalization이 병합할 수 있게 연속된다.

ALGORITHM A26 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** 같은 recovered UUID를 중복 삽입하지 않는다.
- **내구성·복구의 질문으로 읽기.** assistant sibling이 tool_result보다 먼저 온다.
- **상태·계산 모델의 질문으로 읽기.** group은 API normalization이 병합할 수 있게 연속된다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- timestamp tie/clock anomaly
- message.id 부재
- 새 DAG pattern 미포함

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 이미 기록된 legacy topology를 write migration 없이 read-side repair한다.

RECOMMENDATION 새 write format에는 explicit turn/tool group record를 추가하고 legacy만 recovery pass로 처리한다.

ALGORITHM A26 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

기본 chain 밖의 병렬 tool results를 group해 assistant anchor 뒤에 삽입한다.

effect가 시작되고 관찰되는 경계

disk record는 바뀌지 않지만 model이 보는 replay sequence가 보정된다.

현재 설계가 선택한 trade-off와 비용

이미 기록된 legacy topology를 write migration 없이 read-side repair한다. 이 선택의 계산·운영 비용은 index O(N), group별 sort 총합 최악 O(N log N), splice O(N).로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

message.id 부재, 새로운 DAG pattern, timestamp tie는 잘못된 삽입 또는 누락을 만든다. 이때 함께 대입할 반례는 timestamp tie/clock anomaly, message.id 부재, 그리고 새 DAG pattern 미포함이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

복구된 record id, anchor, ordering reason을 ledger로 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A26 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	legacy 병렬 result를 read-side에서 찾아 stable하게 삽입하는 pass가 있다.
CONDITIONAL	알려진 sibling pattern과 anchor가 존재할 때 누락된 result를 복원한다.
UNKNOWN	모든 과거 write topology와 clock anomaly의 분포는 확인되지 않는다.
NEXT READ	새 write format에는 explicit turn/tool group을 추가하고 recovery는 legacy 범위로 한정해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. 같은 timestamp의 두 result 순서를 바꾸면 provider tool pairing 의미가 달라지는가?
2. anchor assistant가 누락됐다면 orphan을 삽입하는 대신 어떤 unknown을 남겨야 하는가?

이 설명을 확인할 코드 src/utils/sessionStorage.ts:2118-2205

ALGORITHM A27 · 독자의 질문

Resume consistency telemetry 은 어떤 시스템 문제를 닫는가

복구 함수가 조용히 record를 잃어도 resume 화면은 그럴듯해 보일 수 있다. disk replay와 in-memory history의 개수 차이를 checkpoint에서 관측해 회귀를 드러낸다.

코드가 필요해지는 이유

in-memory history와 disk replay가 서로 다른 message 집합을 만드는 회귀를 운영 중 탐지한다.

OBSERVED — checkResumeConsistency가 reconstructed messages에서 최신 turn_duration checkpoint를 찾아 비교 신호를 만든다.

INFERENCE — 복구 알고리즘 바깥의 운영 관측이며 chain을 직접 고치지 않는다.

이 절에서 답할 결정 질문

telemetry sentinel은 무엇을 발견할 수 있고 왜 자체로 repair correctness를 증명하지 못하는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **round-trip invariant와 sentinel.** write 전후의 의미가 같아야 한다는 invariant를 중간 checkpoint count로 근사한다. sentinel 부재나 count 의미 변경은 관측 자체를 무효화한다.
>
> 2208–2243행에서 어떤 checkpoint와 count를 비교하고 mismatch를 어떻게 기록하는지 본다.

이 설명을 확인할 코드 src/utils/sessionStorage.ts:2208-2243

ALGORITHM A27 · 실제 코드

OBSERVED

Resume consistency telemetry 의 결정점은 어디에 있는가

저장 당시 messageCount와 replay 위치를 비교해 round-trip drift를 측정한다.

입력 표현. reconstructed Message[]

보존되는 작업 상태. latest turn_duration checkpoint

checkResumeConsistency

src/utils/sessionStorage.ts:2208-2243

```
2208  /**
2209   * Find the latest turn_duration checkpoint in the reconstructed chain and
2210   * compare its recorded messageCount against the chain's position at that
2211   * point. Emits tengu_resume_consistency_delta for BigQuery monitoring of
2212   * write→load round-trip drift — the class of bugs where snip/compact/
2213   * parallel-TR operations mutate in-memory but the parentUuid walk on disk
2214   * reconstructs a different set (adamr-20260320-165831: 397K displayed →
2215   * 1.65M actual on resume).
2216   *
2217   * delta > 0: resume loaded MORE than in-session (the usual failure mode)
2218   * delta < 0: resume loaded FEWER (chain truncation — #22453 class)
2219   * delta = 0: round-trip consistent
2220   *
2221   * Called from loadConversationForResume — fires once per resume, not on
… 19 lines omitted; use the source link for the complete range …
2241      return
2242    }
2243  }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A27 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

reconstructed Message[]은 처음부터 신뢰된 실행 상태가 아니다. latest turn_duration checkpoint가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 뒤에서부터 checkpoint를 찾는다. 이 결과가 다음 판단 "기록된 expected count를 읽는다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 기록된 expected count를 읽는다. 이 결과가 다음 판단 "현재 index를 actual로 계산한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 현재 index를 actual로 계산한다. 이 결과가 다음 판단 "delta와 chain metadata를 telemetry로 남긴다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** delta와 chain metadata를 telemetry로 남긴다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

가장 최근 checkpoint 하나를 사용한다. checkpoint 자신을 제외한 위치 계산을 유지한다. 측정은 history를 수정하지 않는다.

ALGORITHM A27 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **프로토콜·식별의 질문으로 읽기.** 가장 최근 checkpoint 하나를 사용한다.
- **내구성·복구의 질문으로 읽기.** checkpoint 자신을 제외한 위치 계산을 유지한다.
- **검증·관측·운영의 질문으로 읽기.** 측정은 history를 수정하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- checkpoint 없음
- count 의미 변경
- telemetry는 drift를 발견하지만 고치지 않음

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 복구 알고리즘의 correctness를 production observation으로 검증한다.

RECOMMENDATION delta가 큰 경우 repair ledger와 source entry ids를 local diagnostic에 연결한다.

ALGORITHM A27 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

replay 뒤 최신 checkpoint의 기대 count와 실제 reconstructed count를 비교한다.

실패 뒤 남는 상태

checkpoint 부재와 schema 의미 변화는 false confidence 또는 false alarm을 만든다.

OBSERVED	resume consistency count를 비교하는 telemetry가 있다.
CONDITIONAL	checkpoint가 있고 count semantics가 동일할 때 drift 신호가 의미 있다.
UNKNOWN	backend 수집·alert·대응 절차는 source snapshot에서 확인되지 않는다.
NEXT READ	count mismatch가 큰 사례를 A25/A26 repair ledger와 연결해야 한다.

다시 적용할 질문

1. message count가 같아도 lineage가 다르면 이 sentinel이 잡을 수 있는가?
2. checkpoint schema가 바뀌면 구버전 transcript와 어떤 version gate가 필요한가?

DISTINCTION TABLE

OBSERVED

Journal·workspace·protocol truth

resume 시 세 진실을 하나가 다른 하나로 덮어쓰지 않는다.

Truth	알려 주는 것	모르는 것
Journal	과거 intent와 관찰 순서	effect의 현재 지속 여부
Workspace	현재 file/process/task	왜 그렇게 되었는지
Protocol	provider가 수용할 message pairing	실제 OS effect

이 표를 읽는 관점

reconciliation은 세 관찰을 비교하고 repair 사실을 남기는 과정이다.

SELECTIVE EXECUTION TRACE

OBSERVED

사례: Edit는 commit됐지만 result append 전에 종료

4권의 committed-unobserved 상태를 resume한다.

기록·복구

Journal

tool_use e1은 있으나 result가 없다.

이때 남은 상태와 판단이 “Workspace”에서 사용할 전제가 된다.

책임 연결

Workspace

config.ts는 이미 new content다.

이때 남은 상태와 판단이 “Detection”에서 사용할 전제가 된다.

책임 연결

Detection

unresolved tool use와 before/after evidence를 비교한다.

이때 남은 상태와 판단이 “Repair”에서 사용할 전제가 된다.

책임 연결

Repair

재실행하지 않고 synthetic/recovered observation을 만든다.

이때 남은 상태와 판단이 “Continue”에서 사용할 전제가 된다.

책임 연결

Continue

모델은 repair 사실을 포함한 history를 받는다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

동일 e1을 blindly retry하지 않는다.

어디에서 설명이 틀어지는가

workspace read-back 없이 error result만 만들면 실제 변경을 숨긴다.

이 설명을 확인할 코드 src/utils/sessionStorage.ts:4478-4525

02 Compaction과 cancellation

정보를 줄이거나 실행을 멈출 때 이미 발생한 사실을 보존해야 한다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	책임 연결 AbortController는 협력적 broadcast다	알고리즘	책임 연결 Cooperative cancellation propagation
구분	기록·복구 Compaction에서 보존해야 할 정보	알고리즘	상태 소유 Post-compaction context restoration
해석	책임 연결 Cancellation과 rollback을 구분한다	사례	관찰·정산 중단 시 settlement checklist

코드 독해의 시작점

`src/services/compact/**` → `src/utils/abortController.ts` → `src/QueryEngine.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

AbortController는 협력적 broadcast다

signal은 요청 상태를 전파하지만 외부 effect를 되돌리지 않는다.

session abort ownership

src/QueryEngine.ts:184-205

```
184  export class QueryEngine {
185    private config: QueryEngineConfig
186    private mutableMessages: Message[]
187    private abortController: AbortController
188    private permissionDenials: SDKPermissionDenial[]
189    private totalUsage: NonNullableUsage
190    private hasHandledOrphanedPermission = false
191    private readFileState: FileStateCache
192    // Turn-scoped skill discovery tracking (feeds was_discovered on
193    // tengu_skill_tool_invocation). Must persist across the two
194    // processUserInputContext rebuilds inside submitMessage, but is cleared
195    // at the start of each submitMessage to avoid unbounded growth across
196    // many turns in SDK mode.
197    private discoveredSkillNames = new Set<string>()
198    private loadedNestedMemoryPaths = new Set<string>()
199
200    constructor(config: QueryEngineConfig) {
201      this.config = config
202      this.mutableMessages = config.initialMessages ?? []
203      this.abortController = config.abortController ?? createAbortController()
204      this.permissionDenials = []
205      this.readFileState = config.readFileCache
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	controller는 engine/session object가 소유한다.
그 표현이 만든 조건	tool context에 controller가 전달된다.
그 표현이 만든 조건	새 submit에서 reset/교체 시점을 확인해야 한다.

언어와 런타임은 여기까지 보장한다

AbortSignal은 listener와 I/O API가 관찰할 때만 작업을 멈춘다.

프로젝트가 이 메커니즘에 맡긴 책임

파일 write가 끝난 뒤 abort되면 settlement가 실제 state를 기록해야 한다.

경계를 잘못 읽을 때 생기는 위험

UI의 “중단됨”을 “변경 없음”으로 표현하면 안 된다.

이 설명을 확인할 코드 src/QueryEngine.ts:184-205

ALGORITHM A28 · 독자의 질문

Cooperative cancellation propagation 은 어떤 시스템 문제를 닫는가

Node runtime에서 진행 중 작업을 강제로 끊으면 child process와 finally가 정산되지 않을 수 있다. signal을 전파해 각 phase가 안전한 중단 지점에서 멈추게 해야 한다.

코드가 필요해지는 이유

강제 thread termination 없이 진행 중 turn을 중단하고 cleanup 기회를 보장한다.

OBSERVED tool execution이 AbortSignal, in-progress tool ids, aborted/terminal observation을 phase별로 다룬다.

INFERENCE 상위 turn이 취소 의도를 소유하고 각 executor가 자신의 resource cleanup을 소유한다.

이 절에서 답할 결정 질문

abort 요청, 실제 effect 중단, generator terminal, cleanup 완료는 어떤 서로 다른 상태인가?

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:413-468

ALGORITHM A28 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/tools/toolExecution.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

협력적 취소와 structured concurrency

signal은 중단 명령이 아니라 중단 요청이다. child 작업이 signal을 관찰하고 종료하며 parent가 settlement를 기다릴 때 lifetime이 구조화된다.

> **실제 코드로 돌아가면.** 413–468행에서 abort gate와 실행 중/후 cleanup이 어떤 terminal result를 만드는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

동시성·시간·합의
`structured concurrency · cooperative cancellation · settlement`
stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

ALGORITHM A28 · 실제 코드

OBSERVED

Cooperative cancellation propagation의 결정점은 어디에 있는가

하나의 abort intent를 provider, tool, child task가 각 safe point에서 관찰한다.

입력 표현. AbortSignal, 그리고 현재 phase/effect state

보존되는 작업 상태. aborted flag, in-progress tool ids, 그리고 terminal observation

tool execution abort gate

src/services/tools/toolExecution.ts:413-468

```
413     const toolInput = toolUse.input as { [key: string]: string }
414     try {
415       if (toolUseContext.abortController.signal.aborted) {
416         logEvent('tengu_tool_use_cancelled', {
417           toolName: sanitizeToolNameForAnalytics(tool.name),
418           toolUseID:
419             toolUse.id as AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
420           isMcp: tool.isMcp ?? false,
421
422           queryChainId: toolUseContext.queryTracking
423             ?.chainId as AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
424           queryDepth: toolUseContext.queryTracking?.depth,
425           ...(mcpServerType && {
426             mcpServerType:
… 39 lines omitted; use the source link for the complete range …
466       )) {
467         yield update
468       }
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A28 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

AbortSignal, 그리고 현재 phase/effect state은 처음부터 신뢰된 실행 상태가 아니다. aborted flag, in-progress tool ids, 그리고 terminal observation가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 새 effect 시작 전 signal을 검사한다. 이 결과가 다음 판단 "취소 result를 같은 tool_use_id로 만든다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 취소 result를 같은 tool_use_id로 만든다. 이 결과가 다음 판단 "I/O API와 child runtime에 signal을 전달한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** I/O API와 child runtime에 signal을 전달한다. 이 결과가 다음 판단 "finally/settlement에서 task와 state를 정리한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** finally/settlement에서 task와 state를 정리한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

abort 후 새 effect를 시작하지 않는다. 이미 commit된 effect를 취소됐다고 지우지 않는다. 모든 시작된 tool id가 terminal 상태로 수렴한다.

ALGORITHM A28 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** abort 후 새 effect를 시작하지 않는다.
- **동시성·시간·합의의 질문으로 읽기.** 이미 commit된 effect를 취소됐다고 지우지 않는다.
- **동시성·시간·합의의 질문으로 읽기.** 모든 시작된 tool id가 terminal 상태로 수렴한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- signal 미전달
- native child process 미종료
- generator close와 abort 불일치

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED Node event loop에서 안전한 cooperative cancellation을 사용한다.

RECOMMENDATION phase별 cancellation receipt와 timeout/escalation 정책을 정의한다.

ALGORITHM A28 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

abort signal이 phase와 tool에 전달되고 executor가 작업을 멈추며 terminal observation을 반환한다.

effect가 시작되고 관찰되는 경계

이미 시작한 filesystem/process/network effect는 별도 cleanup 또는 partial disposition을 남길 수 있다.

현재 설계가 선택한 trade-off와 비용

Node event loop에서 안전한 cooperative cancellation을 사용한다. 이 선택의 계산·운영 비용은 listener fan-out에 선형, 실제 종료 시간은 하위 작업 협력에 의존.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

signal 미전달, native process 잔존, generator close 불일치는 UI만 종료되고 effect는 계속되는 상태를 만든다. 이때 함께 대입할 반례는 signal 미전달, native child process 미종료, 그리고 generator close와 abort 불일치이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

requested, observed, effect-stopped, cleanup-settled 시점을 분리해 기록해야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A28 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	AbortSignal gate와 cooperative tool cancellation 경로가 있다.
CONDITIONAL	모든 하위 executor가 signal을 전달·관찰할 때 parent cancellation이 settle된다.
UNKNOWN	native child와 외부 service가 실제로 얼마나 빨리 멈추는지는 확인되지 않는다.
NEXT READ	phase별 cancellation receipt와 timeout/escalation 정책을 executor별로 대조해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. abort 뒤 tool result가 늦게 도착하면 journal과 UI는 이를 성공으로 투영할 수 있는가?
2. parent iterator close와 child AbortSignal 중 어느 것이 먼저여야 cleanup이 누락되지 않는가?

이 설명을 확인할 코드 src/services/tools/toolExecution.ts:413-468

DISTINCTION TABLE

OBSERVED

Compaction에서 보존해야 할 정보

자연어 summary와 typed state의 역할을 나눈다.

정보	형식	이유
tool_use/result id	typed record	protocol pairing
미완료 task	task state/id	resume reconciliation
결정 이유	summary + provenance	후속 판단
대용량 read	tombstone + path/version	필요 시 재읽기
plan/mode	typed attachment	행동 제약

이 표를 읽는 관점

future action에 필요한 state는 summary 문장에만 맡기지 않는다.

ALGORITHM A29 · 독자의 질문

Post-compaction context restoration 은 어떤 시스템 문제를 닫는가

compaction summary만으로 다음 turn을 시작하면 최근 파일 내용과 active task 같은 실행 전제가 낡거나 빠질 수 있다. 필요한 resource를 다시 관찰해 working set을 재수화한다.

코드가 필요해지는 이유

summary가 실행 전제와 mode를 잃어 다음 turn이 잘못된 world model을 갖는 것을 방지한다.

OBSERVED compact service가 pre-compact history와 read state에서 attachment set과 dedup key를 만든다.

INFERENCE summary 이후 실행 가능한 context를 만드는 checkpoint restore 단계다.

이 절에서 답할 결정 질문

무엇을 restore attachment로 선택하고 stale·missing·permission error를 어떻게 다음 context에 표현하는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **cache rehydration과 working set.** 직렬화하지 않은 파생 상태는 source resource에서 다시 계산한다. 재관찰 시 resource가 바뀔 수 있으므로 과거 값 복사가 아니라 현재 disposition을 남겨야 한다.
>
> 1399–1448행에서 recent file과 task를 선택·dedup하고 read failure를 처리하는 방식을 본다.

이 설명을 확인할 코드 src/services/compact/compact.ts:1399-1448

ALGORITHM A29 · 실제 코드

OBSERVED

Post-compaction context restoration의 결정점은 어디에 있는가

요약 뒤에도 최근 file, plan, skills, async agents를 typed attachment로 재주입한다.

입력 표현. pre-compact history, preserved messages, read state, 그리고 active tasks

보존되는 작업 상태. attachment set, 그리고 dedup keys

recent file restoration

src/services/compact/compact.ts:1399-1448

```
1399   * Creates attachment messages for recently accessed files to restore them after compaction.
1400   * This prevents the model from having to re-read files that were recently accessed.
1401   * Re-reads files using FileReadTool to get fresh content with proper validation.
1402   * Files are selected based on recency, but constrained by both file count and token budget limits.
1403   *
1404   * Files already present as Read tool results in preservedMessages are skipped —
1405   * re-injecting identical content the model can already see in the preserved tail
1406   * is pure waste (up to 25K tok/compact). Mirrors the diff-against-preserved
1407   * pattern that getDeferredToolsDeltaAttachment uses at the same call sites.
1408   *
1409   * @param readFileState The current file state tracking recently read files
1410   * @param toolUseContext The tool use context for calling FileReadTool
1411   * @param maxFiles Maximum number of files to restore (default: 5)
1412   * @param preservedMessages Messages kept post-compact; Read results here are skipped
… 33 lines omitted; use the source link for the complete range …
1446          'compact',
1447        )
1448        return attachment ? createAttachmentMessage(attachment) : null
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A29 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

pre-compact history, preserved messages, read state, 그리고 active tasks은 처음부터 신뢰된 실행 상태가 아니다. attachment set, 그리고 dedup keys가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** preserved message에 이미 있는 read를 찾는다. 이 결과가 다음 판단 "최근 접근 file을 선택한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 최근 접근 file을 선택한다. 이 결과가 다음 판단 "중복 path를 제거한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 중복 path를 제거한다. 이 결과가 다음 판단 "현재 내용을 다시 읽어 attachment를 만든다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 현재 내용을 다시 읽어 attachment를 만든다. 이 결과가 다음 판단 "plan/skill/agent state attachment와 합친다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** plan/skill/agent state attachment와 합친다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

preserved content를 중복 주입하지 않는다. 오래된 read result보다 현재 file truth를 사용한다. attachment origin을 유지한다.

ALGORITHM A29 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **내구성·복구의 질문으로 읽기.** preserved content를 중복 주입하지 않는다.
- **구성·변형·성능의 질문으로 읽기.** 오래된 read result보다 현재 file truth를 사용한다.
- **내구성·복구의 질문으로 읽기.** attachment origin을 유지한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- 파일이 삭제/변경
- 너무 많은 attachment로 재팽창
- read permission 변화

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 모든 tool result를 summary에 넣지 않고 필요한 resource를 다시 관찰한다.

RECOMMENDATION restore budget과 stale/error disposition을 명시적 record로 남긴다.

ALGORITHM A29 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

선택한 최근 resource를 다시 읽어 attachment로 다음 context에 넣는다.

실패 뒤 남는 상태

삭제·변경·과다 attachment·권한 변화는 restore를 부분 상태로 만든다.

OBSERVED	recent file restoration과 attachment dedup 경로가 있다.
CONDITIONAL	resource가 접근 가능하고 budget 안에 있을 때 최신 관찰을 다음 turn에 제공한다.
UNKNOWN	실제 file churn과 restore 실패 빈도는 알 수 없다.
NEXT READ	restore budget과 stale/error disposition을 typed record로 분리해야 한다.

다시 적용할 질문

1. 파일이 compaction 뒤 삭제됐다면 과거 content를 다시 넣는 것과 missing을 알리는 것 중 무엇이 안전한가?
2. 같은 파일의 두 range attachment는 어떤 dedup key를 가져야 하는가?

CONCEPT IN CONTEXT

INFERENCE

Cancellation과 rollback을 구분한다

중단은 미래 작업을 막는 control signal이고 rollback은 이미 commit된 effect를 역변환하는 별도 domain operation이다.

코드에서 읽어낼 수 있는 단서

출발 단서	Bash child process kill은 파일 side effect를 되돌리지 않는다.
연결되는 단서	FileEdit 역변환은 현재 file version을 다시 검사해야 한다.
연결되는 단서	모델 출력 partial을 UI에서 숨겨도 transcript/network 관찰이 사라지지 않는다.

이 단서들을 하나의 설계로 묶으면

saga compensation은 모든 effect에 자동으로 존재하지 않는다.

시스템 이해에 남는 결론

보고서와 UI는 cancelled, compensated, unchanged를 다른 상태로 표현해야 한다.

SELECTIVE EXECUTION TRACE

OBSERVED

중단 시 settlement checklist

phase별로 남겨야 할 최소 evidence를 정한다.

책임 연결

MODEL

partial assistant와 request id

이때 남은 상태와 판단이 “TOOLS-before-call”에서 사용할 전제가 된다.

효과·위임

TOOLS-before-call

cancelled result

이때 남은 상태와 판단이 “TOOLS-after-commit”에서 사용할 전제가 된다.

효과·위임

TOOLS-after-commit

read-back와 committed receipt

이때 남은 상태와 판단이 “CHILD”에서 사용할 전제가 된다.

책임 연결

CHILD

signal delivery와 terminal task status

이때 남은 상태와 판단이 “SESSION”에서 사용할 전제가 된다.

상태 소유

SESSION

journal flush와 unresolved marker

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

중단 후 다음 turn이 무엇을 믿을지 결정할 수 있어야 한다.

어디에서 설명이 틀어지는가

terminal UI 상태만 갱신하고 journal/task가 남으면 resume가 drift 한다.

03 Session memory는 제한된 별도 effect로 대화를 축약한다

대화 history, compaction summary, CLAUDE.md, session memory를 하나의 "기억"으로 부르지 않는다. session memory는 threshold가 선택한 시점에 restricted forked agent가 별도 파일을 갱신하는 파생 상태다.

이 장의 논증은 다음 질문을 차례로 해결한다

알고리즘	상태 소유 Session memory extraction trigger	코드	책임 연결 Memory file setup은 create-if-absent와 cache invalidation을 함께 수행한다
알고리즘	효과·위임 Capability가 축소된 memory extraction agent	코드	책임 연결 bounded wait는 extraction 완료를 보장하지 않고 기다림의 상한만 정한다
구분	책임 연결 네 가지 기억 표현의 권위와 손실	사례	기록·복구 Compaction과 memory extraction이 겹치는 경우

코드 독해의 시작점

`src/services/SessionMemory/sessionMemory.ts` → `src/services/SessionMemory/sessionMemoryUtils.ts` → `src/services/SessionMemory/prompts.ts`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

ALGORITHM A40 · 독자의 질문

Session memory extraction trigger은 어떤 시스템 문제를 닫는가

session memory agent를 매 turn 실행하면 비용이 크고 너무 드물면 최신 correction과 task state를 놓친다. token과 tool-call threshold로 추출 시점을 보수적으로 정한다.

코드가 필요해지는 이유

매 turn마다 보조 agent를 실행하지 않으면서 긴 session의 작업 상태와 교정 내용을 주기적으로 별도 memory에 축약한다.

OBSERVED SessionMemory가 initialized flag, last extraction token count, last memory message UUID를 소유한다.

INFERENCE trigger는 memory 내용을 만들지 않고 별도 extraction effect를 시작할지 결정한다.

이 절에서 답할 결정 질문

초기 추출과 갱신 threshold가 history branch·compaction·동시 caller에서도 같은 generation을 가리키는가?

이 설명을 확인할 코드 src/services/SessionMemory/sessionMemory.ts:134-179

ALGORITHM A40 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/SessionMemory/sessionMemory.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
보수적 scheduling과 snapshot identity

비용과 최신성 사이에서 일정 threshold를 넘을 때만 작업한다. count만 저장하면 history가 rewind되거나 compaction된 뒤 같은 snapshot인지 알기 어렵다.

> **실제 코드로 돌아가면.** 134–179행에서 초기/update/tool threshold와 last UUID가 함께 갱신되는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

동시성·시간·합의
`conservative scheduling · snapshot consistency`
stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

구성·변형·성능
`working set`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

ALGORITHM A40 · 실제 코드

OBSERVED

Session memory extraction trigger의 결정점은 어디에 있는가

token growth, tool-call 수, 자연스러운 turn 경계를 함께 사용해 비싼 memory extraction의 시점을 결정한다.

입력 표현. 현재 Message[], 초기화 token threshold, update token threshold, 그리고 tool-call threshold

보존되는 작업 상태. sessionMemoryInitialized, tokensAtLastExtraction, 그리고 lastMemoryMessageUuid

memory extraction threshold decision

src/services/SessionMemory/sessionMemory.ts:134-179

```
134  export function shouldExtractMemory(messages: Message[]): boolean {
135    // Check if we've met the initialization threshold
136    // Uses total context window tokens (same as autocompact) for consistent behavior
137    const currentTokenCount = tokenCountWithEstimation(messages)
138    if (!isSessionMemoryInitialized()) {
139      if (!hasMetInitializationThreshold(currentTokenCount)) {
140        return false
141      }
142      markSessionMemoryInitialized()
143    }
144
145    // Check if we've met the minimum tokens between updates threshold
146    // Uses context window growth since last extraction (same metric as init threshold)
147    const hasMetTokenThreshold = hasMetUpdateThreshold(currentTokenCount)
… 29 lines omitted; use the source link for the complete range …
177      return true
178    }
179
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A40 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

현재 Message[], 초기화 token threshold, update token threshold, 그리고 tool-call threshold은 처음부터 신뢰된 실행 상태가 아니다. sessionMemoryInitialized, tokensAtLastExtraction, 그리고 lastMemoryMessageUuid가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** 전체 context token 추정치가 initialization threshold를 넘는지 확인한다. 이 결과가 다음 판단 "직전 extraction 이후 token growth를 계산한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** 직전 extraction 이후 token growth를 계산한다. 이 결과가 다음 판단 "lastMemoryMessageUuid 이후 tool call 수를 센다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** lastMemoryMessageUuid 이후 tool call 수를 센다. 이 결과가 다음 판단 "마지막 assistant turn에 tool call이 남았는지 검사한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 마지막 assistant turn에 tool call이 남았는지 검사한다. 이 결과가 다음 판단 "token+tool threshold 또는 token+natural break 조건을 만족하면 마지막 UUID를 새 기준점으로 기록한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** token+tool threshold 또는 token+natural break 조건을 만족하면 마지막 UUID를 새 기준점으로 기록한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

token growth threshold는 두 trigger 경로 모두에서 반드시 필요하다. 초기화 threshold 전에는 extraction을 시작하지 않는다. tool call이 많은 것만으로 extraction을 반복하지 않는다. 기준 UUID는 선택된 동일 history snapshot의 message를 가리킨다.

ALGORITHM A40 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **동시성·시간·합의의 질문으로 읽기.** token growth threshold는 두 trigger 경로 모두에서 반드시 필요하다.
- **구성·변형·성능의 질문으로 읽기.** 초기화 threshold 전에는 extraction을 시작하지 않는다.
- **동시성·시간·합의의 질문으로 읽기.** tool call이 많은 것만으로 extraction을 반복하지 않는다.
- **구성·변형·성능의 질문으로 읽기.** 기준 UUID는 선택된 동일 history snapshot의 message를 가리킨다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- compaction으로 token count가 줄어 threshold 기준이 왜곡
- last UUID가 rewind된 branch에 남음
- token estimator drift
- 두 caller가 동시에 trigger해 중복 extraction

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 정확한 event마다 memory를 쓰는 대신 비용과 최신성 사이의 conservative schedule을 택한다.

RECOMMENDATION history/branch identity와 extraction generation을 state에 포함하고 trigger decision을 pure record로 남겨 replay test한다.

ALGORITHM A40 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

현재 message token/tool count를 마지막 추출 상태와 비교해 extraction을 schedule한다.

effect가 시작되고 관찰되는 경계

참이면 별도 model call과 memory write 비용이 발생한다.

현재 설계가 선택한 trade-off와 비용

정확한 event마다 memory를 쓰는 대신 비용과 최신성 사이의 conservative schedule을 택한다. 이 선택의 계산·운영 비용은 token estimation과 tool-call scan이 message/block 수에 선형이며 보조 agent 실행 비용은 trigger 뒤에만 발생한다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

token count 감소, branch rewind, estimator drift, concurrent trigger는 중복 또는 누락을 만든다. 이때 함께 대입할 반례는 compaction으로 token count가 줄어 threshold 기준이 왜곡, last UUID가 rewind된 branch에 남음, token estimator drift, 그리고 두 caller가 동시에 trigger해 중복 extraction이다. 이들을 모두 하나의 “실패”로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

history id, branch, extraction generation, trigger reason을 함께 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A40 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	초기·갱신 token threshold와 tool-call threshold가 존재한다.
CONDITIONAL	history가 단조롭게 자라고 caller가 직렬화될 때 schedule이 예상대로 동작한다.
UNKNOWN	실제 token estimator 오차와 concurrent caller 빈도는 확인되지 않는다.
NEXT READ	trigger decision을 pure record로 추출하고 branch/compaction replay test를 추가해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. compaction으로 token count가 줄면 다음 extraction이 영원히 미뤄질 수 있는가?
2. 두 caller가 같은 threshold를 동시에 넘으면 generation을 어떻게 단일화할 것인가?

이 설명을 확인할 코드 src/services/SessionMemory/sessionMemory.ts:134-179

SOURCE READING · 1/2

OBSERVED

Memory file setup은 create-if-absent와 cache invalidation을 함께 수행한다

template 생성과 기존 content read 사이에 FileReadTool cache가 file_unchanged를 반환하지 않도록 동일 read-state entry를 지운다.

private memory file setup src/services/SessionMemory/sessionMemory.ts:183-232

```
183   async function setupSessionMemoryFile(
184     toolUseContext: ToolUseContext,
185   ): Promise<{ memoryPath: string; currentMemory: string }> {
186     const fs = getFsImplementation()
187
188     // Set up directory and file
189     const sessionMemoryDir = getSessionMemoryDir()
190     await fs.mkdir(sessionMemoryDir, { mode: 0o700 })
191
192     const memoryPath = getSessionMemoryPath()
193
194     // Create the memory file if it doesn't exist (wx = O_CREAT|O_EXCL)
195     try {
196       await writeFile(memoryPath, '', {
197         encoding: 'utf-8',
198         mode: 0o600,
199         flag: 'wx',
200       })
201       // Only load template if file was just created
202       const template = await loadSessionMemoryTemplate()
203       await writeFile(memoryPath, template, {
204         encoding: 'utf-8',
205         mode: 0o600,
206       })
207     } catch (e: unknown) {
… 20 lines omitted; use the source link for the complete range …
228     logEvent('tengu_session_memory_file_read', {
229       content_length: currentMemory.length,
230     })
231
232     return { memoryPath, currentMemory }
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	directory는 0700으로 만들고 file은 wx·0600으로 최초 생성한다.
그 표현이 만든 조건	EEXIST만 기존 memory로 인정하고 다른 I/O 오류는 전파한다.
그 표현이 만든 조건	현재 extraction용 subagent context의 readFileState에서 path를 삭제한다.
그 표현이 만든 조건	FileReadTool을 통해 다시 읽어 edit precondition과 현재 content를 맞춘다.

언어와 런타임은 여기까지 보장한다

wx flag는 같은 path의 overwrite를 막는 atomic create primitive이지만 이후 template write까지 하나의 transaction으로 묶지는 않는다. FileReadTool cache 삭제는 OS file과 application observation을 다시 정렬한다.

프로젝트가 이 메커니즘에 맡긴 책임

session memory도 일반 FileEdit invariant를 재사용하므로 직접 fs.read로 우회하지 않고 tool read state를 준비한다. permission mode와 cache semantics가 보조 agent에서도 동일하게 적용되게 하는 선택이다.

경계를 잘못 읽을 때 생기는 위험

process가 빈 파일 생성과 template write 사이에 죽으면 존재하지만 초기화되지 않은 memory가 남는다. cache entry를 부모 context에서 지우거나 격리 context를 잘못 선택하면 다음 user turn의 read dedup 의미가 바뀐다.

이 설명을 확인할 코드 src/services/SessionMemory/sessionMemory.ts:183-232

ALGORITHM A41 · 독자의 질문

Capability가 축소된 memory extraction agent은 어떤 시스템 문제를 닫는가

memory 품질을 위해 agent runtime을 재사용하지만 보조 agent가 workspace 전체와 parent context를 바꿀 권한은 없다. 같은 runtime이라도 capability와 state handoff를 축소해야 한다.

코드가 필요해지는 이유

model 기반 summarization이 필요하지만 보조 실행이 workspace의 다른 파일이나 parent state를 임의로 바꾸지 못하게 한다.

OBSERVED — SessionMemory extraction이 sequential lock, isolated setupContext, restricted readFileState와 in-progress marker를 소유한다.

INFERENCE — parent REPL은 추출 요청과 결과 handoff를 소유하고 forked agent는 memory 범위의 계산만 소유한다.

이 절에서 답할 결정 질문

sequential lock, isolated context, restricted read state가 실제로 어떤 권한과 race를 제거하는가?

이 설명을 확인할 코드 src/services/SessionMemory/sessionMemory.ts:272-337 src/services/SessionMemory/sessionMemory.ts:458-478 src/services/SessionMemory/sessionMemoryUtils.ts:76-105

ALGORITHM A41 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/services/SessionMemory/sessionMemory.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01

capability attenuation과 구조화된 task lifetime

child에는 parent보다 좁은 tool·path·context만 전달하고, parent는 child 완료 또는 중단을 정산한다. callback 하나를 제한해도 다른 inherited capability가 남으면 blast radius는 줄지 않는다.

> **실제 코드로 돌아가면.** 272–337행에서 fork 시 복사되는 context와 새로 제한되는 callback을 목록으로 대조한다.

이론은 정답이 아니라 코드에 던질 질문이다

권한·신뢰 경계
`capability attenuation`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

구성·변형·성능
`state handoff`
조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

검증·관측·운영
`structured task lifecycle`
최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

ALGORITHM A41 · 최소 예제

경로 문자열 제한만으로는 같은 파일 identity를 보장하지 못한다

허용 path가 symlink를 통해 다른 target을 가리키는 경우를 본다.

설명을 위한 최소 실행 예제

```
allowed = '/workspace/memory.md'
// memory.md -> /outside/shared.md (symlink)
if (request.path === allowed) write(request.path)
// 문자열은 같지만 effect identity는 다를 수 있음
```

관찰 시점	값과 상태	프로젝트 코드에서의 의미
문자열 검사	요청 path가 allowed와 같음	표면적 capability는 통과한다.
filesystem resolve	target은 outside file	실제 effect boundary가 넓어진다.
receipt	realpath/digest가 없으면	어느 resource가 바뀌었는지 복원하기 어렵다.

예제를 실제 구현에 다시 연결한다

restricted callback뿐 아니라 realpath identity와 inherited tool set을 함께 검토해야 한다.

ALGORITHM A41 · 실제 코드

OBSERVED

Capability가 축소된 memory extraction agent의 결정점은 어디에 있는가

parent context를 복제하지 않고 별도 context와 "정확한 memory file 하나의 Edit만" 허용하는 canUseTool로 보조 agent 권한을 줄인다.

입력 표현. REPLHookContext, currentMemory, memory path, 그리고 threshold state

보존되는 작업 상태. sequential extraction lock, isolated setupContext, restricted readFileState, 그리고 extraction-in-progress marker

sequential extraction과 forked agent

src/services/SessionMemory/sessionMemory.ts:272-337

```
272  const extractSessionMemory = sequential(async function (
273    context: REPLHookContext,
274  ): Promise<void> {
275    const { messages, toolUseContext, querySource } = context
276
277    // Only run session memory on main REPL thread
278    if (querySource !== 'repl_main_thread') {
279      // Don't log this - it's expected for subagents, teammates, etc.
280      return
281    }
282
283    // Check gate lazily when hook runs (cached, non-blocking)
284    if (!isSessionMemoryGateEnabled()) {
285      // Log gate failure once per session (ant-only)
… 49 lines omitted; use the source link for the complete range …
335      cache_read_input_tokens: usage?.cache_read_input_tokens ?? undefined,
336      cache_creation_input_tokens:
337        usage?.cache_creation_input_tokens ?? undefined,
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A41 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

REPLHookContext, currentMemory, memory path, 그리고 threshold state은 처음부터 신뢰된 실행 상태가 아니다. sequential extraction lock, isolated setupContext, restricted readFileState, 그리고 extraction-in-progress marker가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** main REPL thread와 feature gate를 확인한다. 이 결과가 다음 판단 "threshold가 충족되면 extraction started를 기록한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** threshold가 충족되면 extraction started를 기록한다. 이 결과가 다음 판단 "parent ToolUseContext에서 격리된 subagent context를 만든다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** parent ToolUseContext에서 격리된 subagent context를 만든다. 이 결과가 다음 판단 "현재 memory를 읽고 update prompt를 만든다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** 현재 memory를 읽고 update prompt를 만든다. 이 결과가 다음 판단 "forked agent에 cache-safe params와 restricted canUseTool을 전달한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** forked agent에 cache-safe params와 restricted canUseTool을 전달한다. 이 결과가 다음 판단 "usage와 threshold config를 기록하고 token 기준점을 갱신한다."의 전제가 된다.
결과 기록	**표현과 선택지를 좁힌다** usage와 threshold config를 기록하고 token 기준점을 갱신한다. 이 결과가 다음 판단 "finally에서 extraction marker를 정리한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** finally에서 extraction marker를 정리한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

subagent와 teammate query에서는 자동 extraction을 실행하지 않는다. 한 번에 하나의 extraction만 sequential wrapper를 통과한다. forked agent는 지정된 path에 대한 Edit 이외의 tool effect를 얻지 못한다. 실패해도 extraction-in-progress marker는 terminal 상태로 돌아간다.

ALGORITHM A41 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **권한·신뢰 경계의 질문으로 읽기.** subagent와 teammate query에서는 자동 extraction을 실행하지 않는다.
- **검증·관측·운영의 질문으로 읽기.** 한 번에 하나의 extraction만 sequential wrapper를 통과한다.
- **구성·변형·성능의 질문으로 읽기.** forked agent는 지정된 path에 대한 Edit 이외의 tool effect를 얻지 못한다.
- **권한·신뢰 경계의 질문으로 읽기.** 실패해도 extraction-in-progress marker는 terminal 상태로 돌아간다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- path 문자열은 같지만 symlink/realpath identity가 다름
- restricted callback 밖의 inherited capability
- agent가 edit 없이 성공 종료
- 15초 waiter timeout 뒤 compaction과 extraction이 경쟁

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED 요약 품질을 위해 agent runtime을 재사용하되 capability attenuation과 context isolation으로 blast radius를 제한한다.

RECOMMENDATION memory path를 realpath/file identity로 고정하고 extraction result에 before/after digest와 last summarized UUID를 원자적으로 기록한다.

ALGORITHM A41 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

추출을 직렬화하고 축소한 context로 child agent를 실행해 memory result를 받아 쓴다.

effect가 시작되고 관찰되는 경계

model call과 memory file write가 발생하며 timeout 뒤에도 child가 settle되지 않았을 수 있다.

현재 설계가 선택한 trade-off와 비용

요약 품질을 위해 agent runtime을 재사용하되 capability attenuation과 context isolation으로 blast radius를 제한한다. 이 선택의 계산·운영 비용은 threshold scan O(M) + 별도 model/tool turn; sequential serialization 때문에 대기 시간은 이전 extraction에 의존한다.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

symlink identity, inherited capability, edit 없는 성공, waiter timeout 경쟁은 parent가 잘못 완료를 판단하게 한다. 이때 함께 대입할 반례는 path 문자열은 같지만 symlink/realpath identity가 다름, restricted callback 밖의 inherited capability, agent가 edit 없이 성공 종료, 그리고 15초 waiter timeout 뒤 compaction과 extraction이 경쟁이다. 이들을 모두 하나의 “실패”로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

authority set, child task id, before/after digest, last summarized UUID, settlement를 receipt로 남겨야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A41 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	sequential extraction과 forked agent, 제한된 read state가 구현돼 있다.
CONDITIONAL	제한되지 않은 다른 capability가 없고 path가 실제 file identity와 일치할 때 blast radius가 좁다.
UNKNOWN	feature variant의 전체 inherited tool set과 timeout 후 실제 settlement는 확인되지 않는다.
NEXT READ	realpath/file identity와 atomic write receipt를 추가하기 전 fork context 전체를 inventory해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. child가 성공 종료했지만 memory file digest가 같다면 완료로 볼 수 있는가?
2. 15초 waiter가 끝난 뒤 compaction이 시작되면 extraction과 어떤 state를 경쟁하는가?

이 설명을 확인할 코드 src/services/SessionMemory/sessionMemory.ts:272-337 src/services/SessionMemory/sessionMemory.ts:458-478 src/services/SessionMemory/sessionMemoryUtils.ts:76-105

SOURCE READING · 1/2

OBSERVED

bounded wait는 extraction 완료를 보장하지 않고 기다림의 상한만 정한다

compaction 같은 consumer는 in-progress marker를 poll하지만 stale 1분 또는 wait 15초에 도달하면 memory가 끝나지 않았어도 계속 진행한다.

memory extraction wait policy src/services/SessionMemory/sessionMemoryUtils.ts:76-105

```
76    }
77
78    /**
79     * Mark extraction as completed (called from sessionMemory.ts)
80     */
81    export function markExtractionCompleted(): void {
82      extractionStartedAt = undefined
83    }
84
85    /**
86     * Wait for any in-progress session memory extraction to complete (with 15s timeout)
87     * Returns immediately if no extraction is in progress or if extraction is stale (>1min old).
88     */
89    export async function waitForSessionMemoryExtraction(): Promise<void> {
90      const startTime = Date.now()
91      while (extractionStartedAt) {
92        const extractionAge = Date.now() - extractionStartedAt
93        if (extractionAge > EXTRACTION_STALE_THRESHOLD_MS) {
94          // Extraction is stale, don't wait
95          return
96        }
97
98        if (Date.now() - startTime > EXTRACTION_WAIT_TIMEOUT_MS) {
99          // Timeout - continue anyway
100         return
101       }
102
103       await sleep(1000)
104     }
105   }
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	extractionStartedAt가 없으면 즉시 반환한다.
그 표현이 만든 조건	marker age가 stale threshold를 넘으면 기다리지 않는다.
그 표현이 만든 조건	consumer wait timeout을 넘겨도 오류 대신 반환한다.
그 표현이 만든 조건	1초 polling은 condition variable이 아니라 eventual observation이다.

언어와 런타임은 여기까지 보장한다

polling loop의 return은 "완료됨"과 "더 기다리지 않음"을 같은 void channel로 표현한다. timeout은 liveness를 보호하지만 caller가 freshness를 따로 확인하지 않으면 stale memory를 완료된 memory로 오해할 수 있다.

프로젝트가 이 메커니즘에 맡긴 책임

프로젝트는 compaction latency를 무한히 memory agent에 종속시키지 않는다. 대신 session memory는 authoritative journal이 아니라 best-effort derived state라는 경계를 받아들인다.

경계를 잘못 읽을 때 생기는 위험

process clock 변화, marker cleanup 누락, extraction이 timeout 직후 file을 쓰는 race에서 compaction summary와 memory content가 서로 다른 history cutoff를 가질 수 있다.

이 설명을 확인할 코드 src/services/SessionMemory/sessionMemoryUtils.ts:76-105

DISTINCTION TABLE

OBSERVED

네 가지 기억 표현의 권위와 손실

모두 과거를 담지만 같은 source of truth가 아니다.

표현	생성 방식	권위와 손실
Transcript journal	message append와 parent identity	가장 상세하지만 partial write·replay repair 필요
Compaction summary	현재 turn을 위한 model compression	context budget을 위해 의도적 손실
Session memory	threshold 기반 restricted agent edit	best-effort 파생 파일·bounded stale 가능
CLAUDE.md/instructions	사용자·프로젝트가 제공한 durable instruction	대화 요약이 아니며 precedence·trust 별도

이 표를 읽는 관점

session memory를 transcript 복구 자료나 instruction authority로 자동 승격하지 않는다. 어떤 consumer가 어느 cutoff의 표현을 읽었는지 함께 기록해야 한다.

SELECTIVE EXECUTION TRACE

OBSERVED

Compaction과 memory extraction이 겹치는 경우

두 lossy projection이 같은 history를 서로 다른 시점에 읽을 때 필요한 판단을 본다.

Trigger

token growth가 memory extraction과 auto-compaction 조건을 모두 만족한다.

이때 남은 상태와 판단이 "Extraction"에서 사용할 전제가 된다.

책임 연결

Extraction

forked agent가 기준 UUID까지 memory file을 갱신하기 시작한다.

이때 남은 상태와 판단이 "Wait"에서 사용할 전제가 된다.

책임 연결

Wait

compaction consumer가 bounded wait로 완료를 기다린다.

이때 남은 상태와 판단이 "Timeout branch"에서 사용할 전제가 된다.

책임 연결

Timeout branch

15초 뒤에도 진행 중이면 기존 memory 또는 memory 없는 상태로 compaction을 계속한다.

이때 남은 상태와 판단이 "Late write"에서 사용할 전제가 된다.

책임 연결

Late write

extraction이 뒤늦게 끝나면 다음 turn consumer가 새 memory를 보게 된다.

이때 남은 상태와 판단이 "Reconcile"에서 사용할 전제가 된다.

책임 연결

Reconcile

last summarized UUID와 compact boundary로 두 projection의 cutoff를 비교한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

어느 summary가 더 최신인지 content만 보고 추론하지 않고 history identity와 extraction generation으로 판단해야 한다.

어디에서 설명이 틀어지는가

timeout return을 success receipt로 취급하면 compaction이 아직 쓰이는 file을 authoritative context로 포함할 수 있다.

이 설명을 확인할 코드 src/services/SessionMemory/sessionMemoryUtils.ts:76-105 src/services/SessionMemory/sessionMemory.ts:272-337

04 Agent와 Task는 중첩 runtime이다

child agent는 함수 호출이 아니라 독립 message loop와 lifecycle을 가진다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	효과·위임 Task status는 lifecycle algebra다	알고리즘	진입 경계 Child agent runtime spawn과 event projection
알고리즘	효과·위임 Incomplete child tool call filtering	구분	효과·위임 Parent·Task·Child truth
사례	책임 연결 사례: child가 config.ts를 수정하고 parent가 중단됨	해석	진입 경계 권장 replay test suite

코드 독해의 시작점

`src/tools/AgentTool/runAgent.ts` → `src/Task.ts` → `src/tasks/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Task status는 lifecycle algebra다

pending·running·terminal을 명시적으로 구분해 UI와 cleanup이 같은 상태를 보게 한다.

Task type/status/base src/Task.ts:1-55

```
import { randomBytes } from 'crypto'
import type { AppState } from './state/AppState.js'
import type { AgentId } from './types/ids.js'
import { getTaskOutputPath } from './utils/task/diskOutput.js'

export type TaskType =
  | 'local_bash'
  | 'local_agent'
  | 'remote_agent'
  | 'in_process_teammate'
  | 'local_workflow'
  | 'monitor_mcp'
  | 'dream'

export type TaskStatus =
  | 'pending'
  | 'running'
  | 'completed'
  | 'failed'
  | 'killed'

/**
 * True when a task is in a terminal state and will not transition further.
 * Used to guard against injecting messages into dead teammates, evicting
 * finished tasks from AppState, and orphan-cleanup paths.
… 25 lines omitted; use the source link for the complete range …
  startTime: number
  endTime?: number
  totalPausedMs?: number
  outputFile: string
  outputOffset: number
```

> 이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	TaskType과 TaskStatus union
그 표현이 만든 조건	terminal predicate
그 표현이 만든 조건	AbortController
그 표현이 만든 조건	timestamps와 result

언어와 런타임은 여기까지 보장한다

tagged lifecycle state는 illegal transition을 자동 방지하지 않으므로 reducer/transition guard가 필요하다.

프로젝트가 이 메커니즘에 맡긴 책임

Task는 background execution의 durable projection이지 child engine 그 자체가 아니다.

경계를 잘못 읽을 때 생기는 위험

status만 terminal이고 resource가 남는 settlement gap이 생길 수 있다.

이 설명을 확인할 코드 src/Task.ts:1-55

ALGORITHM A30 · 독자의 질문

Child agent runtime spawn과 event projection은 어떤 시스템 문제를 닫는가

긴 하위 작업을 child agent에 넘기면 parent context는 작아지지만 child effect와 transcript, cancellation이 새로운 nested runtime을 만든다.

코드가 필요해지는 이유

복잡한 하위 작업을 독립 context와 tool set으로 실행하면서 parent authority와 cancellation을 보존한다.

OBSERVED — runAgent가 child messages, abort/task identity, progress/result와 child transcript를 소유한다.

INFERENCE — parent tool call이 spawn authority를 소유하고 child runtime이 격리된 query/tool loop를 소유한다.

이 절에서 답할 결정 질문

parent authority를 줄여 전달하면서 child progress/result와 settlement를 어떤 identity로 다시 합치는가?

이 설명을 확인할 코드 src/tools/AgentTool/runAgent.ts:500-770 src/tools/AgentTool/runAgent.ts:748-790

ALGORITHM A30 · 선행 지식

문법 이름보다 먼저, 이 코드가 기대하는 실행 모델을 만든다

선행 지식은 용어 사전이 아니다. 아래 설명은 작은 실행 모델을 만든 다음, 그 모델이 src/tools/AgentTool/runAgent.ts의 결정점에서 어떤 오독을 막는지 되돌려 확인한다.

01
중첩 상태기계와 structured task lifecycle

parent의 한 tool state 안에 child의 여러 turn 상태가 들어간다. parent terminal은 child가 완료·실패·취소로 settle되고 결과가 projection된 뒤여야 한다.

> **실제 코드로 돌아가면.** 500–770행에서 controller, context, transcript, query event가 어느 task id로 합류하는지 본다.

이론은 정답이 아니라 코드에 던질 질문이다

상태·계산 모델
`nested state machine`
단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

검증·관측·운영
`structured task lifecycle`
최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

권한·신뢰 경계
`capability attenuation`
누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

ALGORITHM A30 · 실제 코드

OBSERVED

Child agent runtime spawn과 event projection의 결정점은 어디에 있는가

AgentTool은 parent context를 투영해 별도의 query loop를 열고 bounded result를 parent로 돌려준다.

입력 표현. agent definition, prompt, parent ToolUseContext, 그리고 model/tools/budget

보존되는 작업 상태. child messages, child abort/task identity, 그리고 progress/result

child controller·context·transcript·query 합류

src/tools/AgentTool/runAgent.ts:500-770

```
500     const resolvedTools = useExactTools
501       ? availableTools
502       : resolveAgentTools(agentDefinition, availableTools, isAsync).resolvedTools
503
504     const additionalWorkingDirectories = Array.from(
505       appState.toolPermissionContext.additionalWorkingDirectories.keys(),
506     )
507
508     const agentSystemPrompt = override?.systemPrompt
509       ? override.systemPrompt
510       : asSystemPrompt(
511           await getAgentSystemPrompt(
512             agentDefinition,
513             toolUseContext,
… 254 lines omitted; use the source link for the complete range …
768         }
769
770         // Yield attachment messages (e.g., structured_output) without recording them
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A30 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

agent definition, prompt, parent ToolUseContext, 그리고 model/tools/budget은 처음부터 신뢰된 실행 상태가 아니다. child messages, child abort/task identity, 그리고 progress/result가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** agent config와 MCP/tool set을 준비한다. 이 결과가 다음 판단 "parent context에서 허용된 state를 child로 투영한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** parent context에서 허용된 state를 child로 투영한다. 이 결과가 다음 판단 "child query()를 실행한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** child query()를 실행한다. 이 결과가 다음 판단 "recordable message를 sidechain에 기록한다."의 전제가 된다.
결정 확정	**표현과 선택지를 좁힌다** recordable message를 sidechain에 기록한다. 이 결과가 다음 판단 "progress를 parent task/UI에 투영한다."의 전제가 된다.
효과 조정	**표현과 선택지를 좁힌다** progress를 parent task/UI에 투영한다. 이 결과가 다음 판단 "terminal summary/result를 합류시킨다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** terminal summary/result를 합류시킨다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

child authority가 parent보다 조용히 넓어지지 않는다. child message history와 parent summary를 구분한다. cancel 시 child task와 signal이 수렴한다.

ALGORITHM A30 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **상태·계산 모델의 질문으로 읽기.** child authority가 parent보다 조용히 넓어지지 않는다.
- **권한·신뢰 경계의 질문으로 읽기.** child message history와 parent summary를 구분한다.
- **검증·관측·운영의 질문으로 읽기.** cancel 시 child task와 signal이 수렴한다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- context 과다 복사
- child effect는 commit됐지만 parent result 누락
- sidechain과 task status drift

이 반례가 나타나면 “알고리즘이 틀렸다”로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED parent context를 오염시키지 않고 긴 하위 탐색을 격리한다.

RECOMMENDATION spawn receipt에 authority, budget, source revision, settlement condition을 명시한다.

ALGORITHM A30 · effect와 운영

정상 경로 뒤에 남는 effect와 실패 뒤에 남는 상태를 구분한다

agent definition과 축소한 context로 child controller를 만들고 query event를 parent progress/result로 투영한다.

effect가 시작되고 관찰되는 경계

child는 허용된 tool로 외부 effect를 만들고 별도 transcript를 남길 수 있다.

현재 설계가 선택한 trade-off와 비용

parent context를 오염시키지 않고 긴 하위 탐색을 격리한다. 이 선택의 계산·운영 비용은 별도 query runtime 비용 + context projection/summary.로 기술되지만, 비용 표기만으로 중복 effect나 부분 상태의 의미가 해결되지는 않는다.

실패·중단 뒤에 실제로 남을 수 있는 상태

context 과다 복사, committed effect 뒤 parent result 누락, status drift는 parent가 재시도하게 만들 수 있다. 이때 함께 대입할 반례는 context 과다 복사, child effect는 commit됐지만 parent result 누락, 그리고 sidechain과 task status drift이다. 이들을 모두 하나의 "실패"로 접으면 retry와 cleanup이 무엇을 이어받아야 하는지 알 수 없다.

관측이 다음 판단에 답하게 하는 법

spawn authority/budget/revision, child task id, effect receipts, settlement reason을 묶어야 한다. 로그의 존재보다 중요한 것은 이 관측값이 같은 요청·state·effect를 가리키며, 복구 또는 중단 결정을 실제로 바꿀 수 있는가이다.

ALGORITHM A30 · 현재 상태 장부

고정 snapshot에서 확인된 것과 아직 말할 수 없는 것을 함께 남긴다

이 장부는 앞의 내용을 압축한 요약이 아니라, 다음 독자가 현재 시스템을 다시 구성하기 위한 출발 상태다.

OBSERVED	child controller·context·transcript·query를 조립하는 runtime path가 있다.
CONDITIONAL	전달 capability와 cancellation이 모든 child executor에 적용될 때 격리가 유지된다.
UNKNOWN	외부 effect의 compensation과 모든 sidechain/status 동기화는 이 범위로 확정되지 않는다.
NEXT READ	spawn receipt와 settlement condition을 명시하기 전 parent/child status writer를 대조해야 한다.

다른 입력과 변경안에 전이해 볼 질문

1. child effect는 commit됐지만 parent result가 끊기면 같은 AgentTool 재시도를 어떻게 식별할 것인가?
2. parent abort 뒤 child transcript의 마지막 partial message는 어디까지 보존해야 하는가?

이 설명을 확인할 코드 src/tools/AgentTool/runAgent.ts:500-770 src/tools/AgentTool/runAgent.ts:748-790

ALGORITHM A31 · 독자의 질문

Incomplete child tool call filtering 은 어떤 시스템 문제를 닫는가

중단된 child transcript에는 tool_use만 있고 result가 없을 수 있다. 그대로 parent model context에 넣으면 provider protocol이 invalid하지만 삭제하면 실패 관찰도 사라진다.

코드가 필요해지는 이유

중단되거나 실패한 child transcript가 provider-invalid pairing을 parent에 만들지 않게 한다.

OBSERVED filterIncompleteToolCalls가 resolved id set과 filtered parent projection을 만든다.

INFERENCE raw child journal은 보존되고 parent context projection만 repair된다.

이 절에서 답할 결정 질문

protocol-valid projection과 raw journal 보존을 어떻게 분리하고 제거된 항목을 어떻게 알리는가?

> ### 이 코드를 읽기 전에 잡을 개념
>
> **참조 무결성과 tombstone.** tool_use와 result의 id relation이 닫혀야 protocol이 valid하다. 제거 사실을 tombstone으로 남기면 projection 손실도 관찰할 수 있다.
>
> 866–905행에서 resolved id를 어떤 branch에서 모으고 어떤 message를 제거하는지 본다.

이 설명을 확인할 코드 src/tools/AgentTool/runAgent.ts:866-905

ALGORITHM A31 · 실제 코드

OBSERVED

Incomplete child tool call filtering의 결정점은 어디에 있는가

child 결과를 합류할 때 unresolved tool_use를 그대로 parent context에 노출하지 않는다.

입력 표현. child Message[]

보존되는 작업 상태. resolved tool id set, 그리고 filtered messages

filterIncompleteToolCalls

src/tools/AgentTool/runAgent.ts:866-905

```
866  export function filterIncompleteToolCalls(messages: Message[]): Message[] {
867    // Build a set of tool use IDs that have results
868    const toolUseIdsWithResults = new Set<string>()
869
870    for (const message of messages) {
871      if (message?.type === 'user') {
872        const userMessage = message as UserMessage
873        const content = userMessage.message.content
874        if (Array.isArray(content)) {
875          for (const block of content) {
876            if (block.type === 'tool_result' && block.tool_use_id) {
877              toolUseIdsWithResults.add(block.tool_use_id)
878            }
879          }
… 23 lines omitted; use the source link for the complete range …
903    })
904  }
905
```

이 발췌는 고정 commit의 실제 코드다. 다음 페이지에서는 문장 순서를 되풀이하지 않고, 각 branch가 상태의 의미를 어떻게 좁히는지 실행 전·중·후로 다시 읽는다.

ALGORITHM A31 · 상태 전이

호출 순서가 아니라, 판단 근거가 어떻게 변하는지를 추적한다

child Message[]은 처음부터 신뢰된 실행 상태가 아니다. resolved tool id set, 그리고 filtered messages가 앞선 관찰을 보존하기 때문에 뒤의 branch가 같은 작업을 계속 판단할 수 있다.

전제 정렬	**진입 조건을 해석한다** tool_result가 존재하는 id를 수집한다. 이 결과가 다음 판단 "assistant content에서 unresolved tool_use block을 제거한다."의 전제가 된다.
표현 정규화	**표현과 선택지를 좁힌다** assistant content에서 unresolved tool_use block을 제거한다. 이 결과가 다음 판단 "빈 message를 정리한다."의 전제가 된다.
유효성 판정	**표현과 선택지를 좁힌다** 빈 message를 정리한다. 이 결과가 다음 판단 "완결된 history만 result projection에 사용한다."의 전제가 된다.
상태 인계	**후속 owner가 읽을 상태를 남긴다** 완결된 history만 result projection에 사용한다. 이 결과를 다음 owner가 읽을 수 있을 때 이 절차가 닫힌다.

이 전이가 끝난 뒤 기대할 수 있는 조건

resolved pair는 유지한다. 불완전 call을 성공으로 만들지 않는다. 원본 sidechain evidence를 삭제하지 않는다.

ALGORITHM A31 · 설명의 보장 경계

이 구현이 지키려는 조건과, 설명이 멈춰야 하는 조건

여기서 불변조건은 형식적 증명의 결론이 아니다. 어느 전이가 의도한 의미를 보존했는지 읽고, 어떤 반례를 넣어 설명의 범위를 확인할지를 정하는 학습 도구다.

전이 전후에 확인할 조건

- **내구성·복구의 질문으로 읽기.** resolved pair는 유지한다.
- **프로토콜·식별의 질문으로 읽기.** 불완전 call을 성공으로 만들지 않는다.
- **내구성·복구의 질문으로 읽기.** 원본 sidechain evidence를 삭제하지 않는다.

최종 출력만 같아도 중간 identity나 effect가 달라질 수 있으므로, 각 조건을 보존하는 transition을 함께 관찰해야 한다.

설명의 범위를 드러내는 반례

- result가 다른 branch에 있어 false unresolved
- filter가 failure observation도 제거
- id collision

이 반례가 나타나면 "알고리즘이 틀렸다"로 끝내지 않고 어떤 state owner와 boundary에서 의미가 갈라졌는지 기록한다.

OBSERVED parent model context의 protocol validity를 우선하면서 raw child journal은 별도로 보존한다.

RECOMMENDATION filtered 항목을 tombstone/repair note로 parent에 알려 정보 손실을 명시한다.

ALGORITHM A31 · 운영과 현재 상태

작은 변환도 effect와 미확정 경계를 남긴다

child messages에서 result가 있는 id를 찾고 incomplete call을 parent projection에서 제외한다.

실패 뒤 남는 상태

result가 다른 branch에 있거나 id가 충돌하면 valid call을 false unresolved로 제거할 수 있다.

OBSERVED	resolved id set을 이용한 incomplete tool call filter가 있다.
CONDITIONAL	id가 branch 전체에서 unique하고 필요한 result가 입력 배열에 있을 때 정확하다.
UNKNOWN	분기된 child journal의 모든 pairing topology는 확인되지 않는다.
NEXT READ	parent projection에 tombstone을 추가할 때 provider-valid shape를 함께 검증해야 한다.

다시 적용할 질문

1. filter된 tool call이 실제 effect를 만들었다면 parent에게 어떤 최소 실패 사실을 남겨야 하는가?
2. 다른 branch의 같은 id가 resolved set에 들어오면 어떤 call이 잘못 보존되는가?

DISTINCTION TABLE

OBSERVED

Parent·Task·Child truth

세 표현은 같은 객체가 아니다.

표현	소유 정보	동기화 위험
Parent message	요청과 최종 요약	중간 effect 누락
Task record	status/progress/cancel	resource와 status drift
Child sidechain	상세 messages/tool effects	parent가 보지 못함

이 표를 읽는 관점

합류는 child history 전체를 복사하는 것이 아니라 필요한 evidence를 투영하는 일이다.

SELECTIVE EXECUTION TRACE

OBSERVED

사례: child가 config.ts를 수정하고 parent가 중단됨

nested effect의 reconciliation을 연습한다.

기록·복구

Child journal

e1 commit/result를 확인한다.

이때 남은 상태와 판단이 "Workspace"에서 사용할 전제가 된다.

책임 연결

Workspace

현재 config.ts digest를 읽는다.

이때 남은 상태와 판단이 "Task"에서 사용할 전제가 된다.

효과·위임

Task

child terminal/release 상태를 확인한다.

이때 남은 상태와 판단이 "Parent"에서 사용할 전제가 된다.

책임 연결

Parent

result 누락을 repair event로 추가한다.

이때 남은 상태와 판단이 "Resume"에서 사용할 전제가 된다.

책임 연결

Resume

중복 AgentTool 실행 없이 계속한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

lineage보다 effect identity와 resource settlement가 우선한다.

어디에서 설명이 틀어지는가

Task status만 보고 완료를 추론하면 committed-unobserved effect를 놓친다.

CONCEPT IN CONTEXT

RECOMMENDATION

권장 replay test suite

이 snapshot에 공식 test suite가 보이지 않으므로 보고서의 개선안은 executable invariant 중심이어야 한다.

코드에서 읽어낼 수 있는 단서

출발 단서	parallel tool calls round-trip
연결되는 단서	abort after file commit
연결되는 단서	compaction followed by resume
연결되는 단서	child sidechain result loss
연결되는 단서	legacy transcript recovery

이 단서들을 하나의 설계로 묶으면

각 fixture는 write→load→normalize round-trip 후 identity와 terminal 상태를 비교해야 한다.

시스템 이해에 남는 결론

공식 upstream 품질을 평가한다는 뜻이 아니라 이 복원 코드에서 관찰한 핵심 알고리즘을 검증하는 학습용 harness다.

05 Team·Task·Mailbox·Worktree는 협업의 서로 다른 상태 공간이다

child agent 한 번의 실행만으로 multi-agent 제품을 설명할 수 없다. 팀 membership, durable task dependency, process-local message delivery, workspace isolation은 서로 다른 owner와 실패 복구를 가진다.

이 장의 논증은 다음 질문을 차례로 해결한다

코드	효과·위임 Team 생성은 membership과 공유 task namespace를 함께 만든다	코드	상태 소유 Task list는 JSON file 집합과 lock으로 공유 coordination state를 직렬화한다
코드	진입 경계 Mailbox는 process-local selective receive queue다	코드	상태 소유 EnterWorktree는 cwd만 바꾸지 않고 session context cache를 재기준화한다
구분	책임 연결 협업 상태의 canonical owner를 섞지 않는다	사례	책임 연결 협업 작업 하나가 완료됐다고 말하기 위한 reconciliation

코드 독해의 시작점

`src/tools/TeamCreateTool/**` → `src/utils/tasks.ts` → `src/utils/mailbox.ts` → `src/tools/EnterWorktreeTool/**`

각 절은 앞 절의 결론을 다음 절의 전제로 사용한다. 따라서 용어 목록을 외우기보다, 현재 구현이 어떤 상태를 만들고 그 상태가 다음 결정의 조건으로 어떻게 사용되는지 연결해서 읽는다.

SOURCE READING · 1/2

OBSERVED

Team 생성은 membership과 공유 task namespace를 함께 만든다

TeamCreate는 이름을 등록하는 UI 기능이 아니라 leader identity, member metadata, durable team file, task-list namespace를 같은 lifecycle에 묶는다.

team validation, leader identity, durable file과 task list setup

src/tools/TeamCreateTool/TeamCreateTool.ts:80-175

```
80      userFacingName() {
81        return ''
82      },
83
84      get inputSchema(): InputSchema {
85        return inputSchema()
86      },
87
88      isEnabled() {
89        return isAgentSwarmsEnabled()
90      },
91
92      toAutoClassifierInput(input) {
93        return input.team_name
94      },
95
96      async validateInput(input, _context) {
97        if (!input.team_name || input.team_name.trim().length === 0) {
98          return {
99            result: false,
100           message: 'team_name is required for TeamCreate',
101           errorCode: 9,
102         }
103       }
104       return { result: true }
… 66 lines omitted; use the source link for the complete range …
171             cwd: getCwd(),
172             subscriptions: [],
173           },
174         ],
175       }
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	leader는 동시에 하나의 team만 이끌도록 AppState에서 기존 context를 검사한다.
그 표현이 만든 조건	team 이름 충돌은 unique name으로 정규화한다.
그 표현이 만든 조건	leader의 model·session·cwd를 첫 member record에 기록한다.
그 표현이 만든 조건	team file을 쓴 뒤 task list를 reset/ensure하고 leaderTeamName을 등록한다.

언어와 런타임은 여기까지 보장한다

membership file과 task directory는 서로 다른 durable resource다. 둘 사이에 transaction이 없으면 team은 존재하지만 task namespace가 준비되지 않거나, 반대로 오래된 task file이 새 team에 보일 수 있다.

프로젝트가 이 메커니즘에 맡긴 책임

TeamCreate가 leader-side composition을 소유하지만 teammate process spawn·mailbox delivery·Task tool mutation은 별도 module이 소유한다. 하나의 teamContext object가 모든 truth를 대체하지 않는다.

경계를 잘못 읽을 때 생기는 위험

team file write 뒤 task reset이 실패하면 partial creation 상태가 남는다. cleanup registration과 TeamDelete가 어떤 resource를 회수하는지 별도 settlement가 필요하다.

이 설명을 확인할 코드 src/tools/TeamCreateTool/TeamCreateTool.ts:80-175

SOURCE READING · 1/2

OBSERVED

Task list는 JSON file 집합과 lock으로 공유 coordination state를 직렬화한다

task status는 UI array가 아니라 여러 process가 읽고 쓰는 durable record이며 dependency edge와 owner를 가진다.

task schema, signal, lock policy, high-water mark

src/utils/tasks.ts:1-122

```
import { mkdir, readdir, readFile, unlink, writeFile } from 'fs/promises'
import { join } from 'path'
import { z } from 'zod/v4'
import { getIsNonInteractiveSession, getSessionId } from '../bootstrap/state.js'
import { uniq } from './array.js'
import { logForDebugging } from './debug.js'
import { getClaudeConfigHomeDir, getTeamsDir, isEnvTruthy } from './envUtils.js'
import { errorMessage, getErrnoCode } from './errors.js'
import { lazySchema } from './lazySchema.js'
import * as lockfile from './lockfile.js'
import { logError } from './log.js'
import { createSignal } from './signal.js'
import { jsonParse, jsonStringify } from './slowOperations.js'
import { getTeamName } from './teammate.js'
import { getTeammateContext } from './teammateContext.js'

// Listeners for task list updates (used for immediate UI refresh in same process)
const tasksUpdated = createSignal()

/**
 * Team name set by the leader when creating a team.
 * Used by getTaskListId() so the leader's tasks are stored under the team name
 * (matching where tmux/iTerm2 teammates look), not under the session ID.
 */
let leaderTeamName: string | undefined
```

… 92 lines omitted; use the source link for the complete range …

```
      const value = parseInt(content, 10)
      return isNaN(value) ? 0 : value
    } catch {
      return 0
    }
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	status는 pending·in_progress·completed로 제한된다.
그 표현이 만든 조건	blocks와 blockedBy를 모두 보존해 dependency를 양방향으로 투영한다.
그 표현이 만든 조건	같은 process의 UI 갱신은 signal을 쓰지만 mutation 성공을 listener 실패와 분리한다.
그 표현이 만든 조건	여러 agent의 critical section은 async file lock과 retry budget으로 직렬화한다.
그 표현이 만든 조건	high-water mark는 reset 뒤 ID 재사용을 막는다.

언어와 런타임은 여기까지 보장한다

file lock은 각 mutation의 상호 배제를 돕지만 task graph 전체의 의미적 correctness를 자동 보장하지 않는다. blocks와 blockedBy의 대칭, owner가 terminal agent인지, completed task가 실제 effect receipt를 가졌는지는 별도 invariant다.

프로젝트가 이 메커니즘에 맡긴 책임

task list ID는 explicit env, teammate context, team name, session ID 순으로 결정된다. 따라서 같은 file schema라도 namespace resolution이 틀리면 leader와 teammate가 서로 다른 task universe를 본다.

경계를 잘못 읽을 때 생기는 위험

lock timeout, process crash, partial file write, stale owner는 모두 durable coordination failure다. in-process signal은 다른 process에 delivery receipt를 제공하지 않는다.

이 설명을 확인할 코드 src/utils/tasks.ts:1-122 src/utils/tasks.ts:123-220

SOURCE READING · 1/2

OBSERVED

Mailbox는 process-local selective receive queue다

message source와 predicate를 가진 queue/waiter는 teammate·system·tick·task event를 즉시 waiter에게 넘기거나 나중 poll을 위해 보존한다.

queue, waiter, revision과 selective receive src/utils/mailbox.ts:1-67

```
1     import { createSignal } from './signal.js'
2
3     export type MessageSource = 'user' | 'teammate' | 'system' | 'tick' | 'task'
4
5     export type Message = {
6       id: string
7       source: MessageSource
8       content: string
9       from?: string
10      color?: string
11      timestamp: string
12    }
13
14    type Waiter = {
15      fn: (msg: Message) => boolean
16      resolve: (msg: Message) => void
17    }
18
19    export class Mailbox {
20      private queue: Message[] = []
21      private waiters: Waiter[] = []
22      private changed = createSignal()
23      private _revision = 0
24
25      get length(): number {
… 37 lines omitted; use the source link for the complete range …
63        return new Promise<Message>(resolve => {
64          this.waiters.push({ fn, resolve })
65        })
66      }
67
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	send는 revision을 먼저 증가시킨다.
그 표현이 만든 조건	첫 matching waiter가 있으면 queue에 넣지 않고 바로 resolve한다.
그 표현이 만든 조건	poll은 첫 matching queued message를 제거한다.
그 표현이 만든 조건	receive는 queued match가 없을 때 predicate와 resolver를 waiter list에 보존한다.
그 표현이 만든 조건	subscribe signal은 queue 변화의 UI projection만 알린다.

언어와 런타임은 여기까지 보장한다

selective receive는 actor mailbox와 비슷하지만 이 class는 process memory 안에만 있다. revision은 변화 감지용 단조 counter이지 durable message offset이나 exactly-once delivery proof가 아니다.

프로젝트가 이 메커니즘에 맡긴 책임

predicate가 겹치는 waiter 중 배열에서 먼저 발견된 하나가 message를 소유한다. consumer 취소 시 waiter 제거 경로가 없으면 resolver와 predicate가 남아 memory/lifecycle leak이 될 수 있다.

경계를 잘못 읽을 때 생기는 위험

process 종료·restart에서 queue는 사라진다. teammate 간 전달의 durable transport와 ack/retry semantics는 이 class 바깥을 함께 읽어야 하며, 이 source만으로 exactly-once를 주장할 수 없다.

이 설명을 확인할 코드 src/utils/mailbox.ts:1-67

SOURCE READING · 1/2

OBSERVED

EnterWorktree는 cwd만 바꾸지 않고 session context cache를 재기준화한다

worktree 생성 뒤 process cwd, application cwd, original cwd, session storage, system prompt와 memory cache를 함께 갱신해야 새 workspace가 일관되게 보인다.

worktree creation과 session context switch

src/tools/EnterWorktreeTool/EnterWorktreeTool.ts:52-123

```
52    export const EnterWorktreeTool: Tool<InputSchema, Output> = buildTool({
53      name: ENTER_WORKTREE_TOOL_NAME,
54      searchHint: 'create an isolated git worktree and switch into it',
55      maxResultSizeChars: 100_000,
56      async description() {
57        return 'Creates an isolated worktree (via git or configured hooks) and switches the session into it'
58      },
59      async prompt() {
60        return getEnterWorktreeToolPrompt()
61      },
62      get inputSchema(): InputSchema {
63        return inputSchema()
64      },
65      get outputSchema(): OutputSchema {
66        return outputSchema()
67      },
68      userFacingName() {
69        return 'Creating worktree'
70      },
71      shouldDefer: true,
72      toAutoClassifierInput(input) {
73        return input.name ?? ''
74      },
75      renderToolUseMessage,
76      renderToolResultMessage,
… 42 lines omitted; use the source link for the complete range …
119     },
120     mapToolResultToToolResultBlockParam({ message }, toolUseID) {
121       return {
122         type: 'tool_result',
123         content: message,
```

이 발췌는 설명을 위해 재작성한 의사코드가 아니라 고정 commit에서 기계적으로 가져온 실제 줄이다. 긴 범위는 가운데만 생략했으며 링크에서 전체 범위를 확인할 수 있다.

SOURCE READING · 2/2

OBSERVED

문법 아래에 숨은 상태와 책임

앞의 코드는 짧은 문법 표현을 사용하지만, 그 표현이 보장하는 범위와 프로젝트가 기대하는 범위는 같지 않다.

줄을 따라 확인할 관찰점

먼저 볼 표현	이미 이 session이 만든 worktree 안이면 중첩 진입을 거부한다.
그 표현이 만든 조건	현재 cwd가 main repo root가 아니면 canonical root로 먼저 이동한다.
그 표현이 만든 조건	새 worktree를 만든 뒤 process.chdir와 setCwd를 함께 호출한다.
그 표현이 만든 조건	worktree state를 저장하고 system prompt, memory, plan directory cache를 비운다.
그 표현이 만든 조건	result에는 path와 branch를 남겨 이후 Exit/cleanup의 identity로 쓴다.

언어와 런타임은 여기까지 보장한다

cwd는 process API의 한 값이 아니라 command execution, instruction discovery, permission root, statusline, transcript metadata가 공유하는 context key다. cache invalidation을 빠뜨리면 shell은 새 worktree를 보지만 prompt는 이전 CLAUDE.md를 보는 split-brain이 생긴다.

프로젝트가 이 메커니즘에 맡긴 책임

이 도구는 명시적 worktree request에서만 사용해야 하는 control-plane effect이며, 실제 Git worktree 생성·삭제는 utils/worktree와 Exit tool의 별도 lifecycle에 걸친다.

경계를 잘못 읽을 때 생기는 위험

중간 실패로 directory만 생성되고 session state가 저장되지 않거나, remove 전에 uncommitted change/commit을 잘못 판정하면 고립 또는 데이터 손실이 생긴다. worktree는 agent authority를 자동 분리하지 않고 filesystem working copy만 격리한다.

이 설명을 확인할 코드 src/tools/EnterWorktreeTool/EnterWorktreeTool.ts:52-123

DISTINCTION TABLE

OBSERVED

협업 상태의 canonical owner를 섞지 않는다

같은 "작업 진행 중"을 나타내더라도 representation마다 답하는 질문이 다르다.

상태 공간	canonical evidence	다른 표현으로 대체할 수 없는 이유
Team membership	team file + session team context	누가 leader/member인지와 task status는 다른 사실
Task graph	task JSON + dependency/owner/status	message 수신만으로 완료·차단 해제를 확정할 수 없음
Mailbox delivery	process-local queue/waiter + 외부 transport state	revision은 durable ack가 아니며 restart에서 사라짐
Agent runtime	child messages, task handle, abort/terminal settlement	task record만으로 external effect와 resource cleanup을 모두 알 수 없음
Workspace isolation	worktree path/branch/head/change state	별도 working copy가 permission·authority 격리를 의미하지 않음

이 표를 읽는 관점

복구는 가장 최신처럼 보이는 표현을 고르는 것이 아니라 동일 agent/task/effect/worktree identity를 연결해 membership, lifecycle, resource settlement를 각각 재검증하는 일이다.

SELECTIVE EXECUTION TRACE

OBSERVED

협업 작업 하나가 완료됐다 고 말하기 위한 reconciliation

leader가 teammate에게 파일 수정을 맡긴 사례에서 서로 다른 truth를 한 번에 닫지 않는다.

책임 연결

Assignment

task record에 owner·blockedBy·expected outcome을 남긴다.

이때 남은 상태와 판단이 "Delivery"에서 사용할 전제가 된다.

책임 연결

Delivery

mailbox/transport가 요청을 전달했는지와 teammate가 어떤 task identity를 읽었는지 확인한다.

이때 남은 상태와 판단이 "Isolation"에서 사용할 전제가 된다.

책임 연결

Isolation

worktree를 썼다면 branch/head/path와 실제 cwd·instruction context가 일치하는지 확인한다.

이때 남은 상태와 판단이 "Effect"에서 사용할 전제가 된다.

효과·위임

Effect

workspace digest·test·commit 또는 변경 receipt로 실제 결과를 관찰한다.

이때 남은 상태와 판단이 "Lifecycle"에서 사용할 전제가 된다.

책임 연결

Lifecycle

child/teammate process와 task status가 terminal로 수렴했는지 확인한다.

이때 남은 상태와 판단이 "Integration"에서 사용할 전제가 된다.

책임 연결

Integration

격리된 변경이 parent workspace에 아직 합류하지 않았다면 "구현 완료, 통합 미완료"로 분리한다.

이 지점에서 앞선 상태 변화가 외부에서 관찰 가능한 결과로 정산된다.

이 흐름 전체에서 보존해야 하는 것

task completed 한 필드나 "done" message 하나는 effect·resource·integration을 모두 증명하지 않는다.

어디에서 설명이 틀어지는가

transport ack, workspace state, agent terminal 중 하나가 UNKNOWN이면 그 차원을 그대로 남기고 중복 effect를 자동 재실행하지 않는다.

이 설명을 확인할 코드 src/utils/tasks.ts:1-122 src/utils/mailbox.ts:1-67 src/tools/EnterWorktreeTool/EnterWorktreeTool.ts:52-123

THEORY ATLAS · 1/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

상태·계산 모델

3개 알고리즘 · 5개 표현

입력이 어떤 중간 표현과 상태 전이를 거쳐 결과가 되는지, 그리고 각 전이 전후에 무엇이 계속 참이어야 하는지를 설명한다.

이 권에서 나타나는 방식. A25 Leaf-to-root conversation chain reconstruction과 A26 병렬 tool result orphan recovery에서 이 관점이 결정적인 이유는 "같은 UUID를 두 번 방문하지 않는다. 또한 같은 recovered UUID를 중복 삽입하지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. linked lineage(A25), cycle detection(A25), DAG linearization(A26), stable sort(A26), nested state machine(A30)

스터디 질문. 단계의 순서나 표현을 바꿔도 같은 상태와 불변조건에 도달하는가?

동시성·시간·합의

2개 알고리즘 · 5개 표현

여러 실행이 같은 상태를 읽고 쓰는 동안 순서, 취소, 재시도, 관찰 시점이 결과의 의미를 바꾸지 않게 하는 조건을 다룬다.

이 권에서 나타나는 방식. A28 Cooperative cancellation propagation과 A40 Session memory extraction trigger에서 이 관점이 결정적인 이유는 "abort 후 새 effect를 시작하지 않는다. 또한 token growth threshold는 두 trigger 경로 모두에서 반드시 필요하다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. structured concurrency(A28), cooperative cancellation(A28), settlement(A28), conservative scheduling(A40), snapshot consistency(A40)

스터디 질문. stale read, 중복 실행, 취소가 끼어들어도 effect와 최종 상태의 의미가 보존되는가?

프로토콜·식별

2개 알고리즘 · 2개 표현

서로 다른 계층이 같은 작업과 같은 메시지를 가리킨다는 사실을 타입, 식별자, completion 규약으로 유지한다.

이 권에서 나타나는 방식. A27 Resume consistency telemetry과 A31 Incomplete child tool call filtering에서 이 관점이 결정적인 이유는 "가장 최근 checkpoint 하나를 사용한다. 또한 resolved pair는 유지한다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. round-trip invariant(A27), referential integrity(A31)

스터디 질문. 생산자와 소비자가 동일한 값·작업·종료 의미를 해석한다는 증거가 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 2/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

내구성·복구

5개 알고리즘 · 7개 표현

부분 기록, 재시작, 버전 변화가 있어도 이전 사실을 잃거나 같은 effect를 다시 확정하지 않도록 만드는 복구 계약을 설명한다.

이 권에서 나타나는 방식. A25 Leaf-to-root conversation chain reconstruction과 A26 병렬 tool result orphan recovery에서 이 관점이 결정적인 이유는 "같은 UUID를 두 번 방문하지 않는다. 또한 같은 recovered UUID를 중복 삽입하지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. event replay(A25), backward-compatible repair(A26), sentinel/checkpoint(A27), cache rehydration(A29), checkpoint restore(A29), projection repair(A31), tombstone(A31)

스터디 질문. 중단 지점에서 재개했을 때 무엇을 재생하고 무엇을 다시 실행하지 않아야 하는가?

권한·신뢰 경계

2개 알고리즘 · 1개 표현

실행 가능성, 허용 여부, 데이터 노출을 분리하고 신뢰하지 않은 입력이 effect 권한으로 곧바로 승격되지 않게 한다.

이 권에서 나타나는 방식. A41 Capability가 축소된 memory extraction agent과 A30 Child agent runtime spawn과 event projection에서 이 관점이 결정적인 이유는 "subagent와 teammate query에서는 자동 extraction을 실행하지 않는다. 또한 child authority가 parent보다 조용히 넓어지지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. capability attenuation(A41·A30)

스터디 질문. 누가 어떤 근거로 effect를 허용하며, 더 약한 권한으로만 위임된다는 사실을 어디서 확인하는가?

구성·변형·성능

3개 알고리즘 · 2개 표현

동일한 핵심 의미를 여러 실행 표면과 제품 변형에 조립하면서 비용과 의존성의 위치를 통제하는 방법을 다룬다.

이 권에서 나타나는 방식. A29 Post-compaction context restoration과 A40 Session memory extraction trigger에서 이 관점이 결정적인 이유는 "preserved content를 중복 주입하지 않는다. 또한 token growth threshold는 두 trigger 경로 모두에서 반드시 필요하다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. working set(A29·A40), state handoff(A41)

스터디 질문. 조립 방식이나 최적화를 바꿔도 핵심 계약이 adapter와 build 변형 안에서 보존되는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

THEORY ATLAS · 3/3

용어를 외우지 않고, 어떤 보장을 설명하는 이론인지 묶어 읽는다

이 지도는 이 권의 알고리즘이 실제로 참조한 이론만 집계한다. 빈도는 중요도 점수가 아니라 여러 구현 결정을 같은 질문으로 비교할 수 있는 탐색 신호다.

검증·관측·운영

3개 알고리즘 · 2개 표현

실행 결과를 관찰하고 회귀를 검출하며, cleanup과 운영 신호가 본래 동작을 왜곡하지 않도록 만드는 증거 체계를 다룬다.

이 권에서 나타나는 방식. A27 Resume consistency telemetry과 A41 Capability가 축소된 memory extraction agent에서 이 관점이 결정적인 이유는 "가장 최근 checkpoint 하나를 사용한다. 또한 subagent와 teammate query에서는 자동 extraction을 실행하지 않는다."라는 조건을 구현 세부사항보다 오래 유지해야 하기 때문이다.

참조 분포. observability(A27), structured task lifecycle(A41·A30)

스터디 질문. 최종 출력뿐 아니라 실패·중단·복구 경로의 보장을 어떤 관찰과 테스트로 입증할 수 있는가?

괄호 안의 알고리즘 ID로 돌아가면 이론의 정의가 아니라 그 이론이 설명하는 source transition, 반례, 불변조건을 함께 확인할 수 있다.

ALGORITHM INDEX

OBSERVED

이 권의 결정적 알고리즘과 보장

알고리즘 이름보다 마지막 열을 먼저 읽는다. 프로젝트에서 중요한 것은 계산 기법 자체보다 그 계산으로 어떤 시스템 조건을 계속 참으로 만드는가이기 때문이다.

ID	문제와 구현	대표 불변조건
A25	**Leaf-to-root conversation chain reconstruction** 한 파일의 여러 branch/metadata 중 선택된 conversation lineage를 복원한다.	같은 UUID를 두 번 방문하지 않는다.
A26	**병렬 tool result orphan recovery** 병렬 tool_use가 만든 DAG를 linked-list traversal만으로 읽을 때 branch가 사라지는 문제를 고친다.	같은 recovered UUID를 중복 삽입하지 않는다.
A27	**Resume consistency telemetry** in-memory history와 disk replay가 서로 다른 message 집합을 만드는 회귀를 운영 중 탐지한다.	가장 최근 checkpoint 하나를 사용한다.
A28	**Cooperative cancellation propagation** 강제 thread termination 없이 진행 중 turn을 중단하고 cleanup 기회를 보장한다.	abort 후 새 effect를 시작하지 않는다.
A29	**Post-compaction context restoration** summary가 실행 전제와 mode를 잃어 다음 turn이 잘못된 world model을 갖는 것을 방지한다.	preserved content를 중복 주입하지 않는다.
A40	**Session memory extraction trigger** 매 turn마다 보조 agent를 실행하지 않으면서 긴 session의 작업 상태와 교정 내용을 주기적으로 별도 memory에 축약한다.	token growth threshold는 두 trigger 경로 모두에서 반드시 필요하다.
A41	**Capability가 축소된 memory extraction agent** model 기반 summarization이 필요하지만 보조 실행이 workspace의 다른 파일이나 parent state를 임의로 바꾸지 못하게 한다.	subagent와 teammate query에서는 자동 extraction을 실행하지 않는다.
A30	**Child agent runtime spawn과 event projection** 복잡한 하위 작업을 독립 context와 tool set으로 실행하면서 parent authority와 cancellation을 보존한다.	child authority가 parent보다 조용히 넓어지지 않는다.
A31	**Incomplete child tool call filtering** 중단되거나 실패한 child transcript가 provider-invalid pairing을 parent에 만들지 않게 한다.	resolved pair는 유지한다.

SOURCE EVIDENCE INDEX · 1/1

OBSERVED

설명을 다시 코드로 되돌리는 출발점

보고서의 해석은 아래 고정 줄 범위에서 다시 검토할 수 있다. 링크는 모두 동일한 commit을 가리키며, 파일 이름만 비슷한 최신 branch로 이동하지 않는다.

고정 코드 범위	연결되는 설명	표현 방식
src/utils/sessionStorage.ts:1408-1465	Transcript write는 chain identity를 남긴다	code
src/utils/sessionStorage.ts:2069-2117	Leaf-to-root conversation chain reconstruction	algorithm
src/utils/sessionStorage.ts:2118-2205	병렬 tool result orphan recovery	algorithm
src/utils/sessionStorage.ts:2208-2243	Resume consistency telemetry	algorithm
src/utils/sessionStorage.ts:4478-4525	사례: Edit는 commit됐지만 result append 전에 종료	walkthrough
src/QueryEngine.ts:184-205	AbortController는 협력적 broadcast다	code
src/services/tools/toolExecution.ts:413-468	Cooperative cancellation propagation	algorithm
src/services/compact/compact.ts:1399-1448	Post-compaction context restoration	algorithm
src/services/SessionMemory/sessionMemory.ts:134-179	Session memory extraction trigger	algorithm
src/services/SessionMemory/sessionMemory.ts:183-232	Memory file setup은 create-if-absent와 cache invalidation을 함께 수행한다	code
src/services/SessionMemory/sessionMemory.ts:272-337	Capability가 축소된 memory extraction agent	algorithm
src/services/SessionMemory/sessionMemory.ts:458-478	Capability가 축소된 memory extraction agent	algorithm
src/services/SessionMemory/sessionMemoryUtils.ts:76-105	Capability가 축소된 memory extraction agent	algorithm
src/Task.ts:1-55	Task status는 lifecycle algebra다	code
src/tools/AgentTool/runAgent.ts:500-770	Child agent runtime spawn과 event projection	algorithm
src/tools/AgentTool/runAgent.ts:748-790	Child agent runtime spawn과 event projection	algorithm
src/tools/AgentTool/runAgent.ts:866-905	Incomplete child tool call filtering	algorithm
src/tools/TeamCreateTool/TeamCreateTool.ts:80-175	Team 생성은 membership과 공유 task namespace를 함께 만든다	code
src/utils/tasks.ts:1-122	Task list는 JSON file 집합과 lock으로 공유 coordination state를 직렬화한다	code
src/utils/tasks.ts:123-220	Task list는 JSON file 집합과 lock으로 공유 coordination state를 직렬화한다	code
src/utils/mailbox.ts:1-67	Mailbox는 process-local selective receive queue다	code
src/tools/EnterWorktreeTool/EnterWorktreeTool.ts:52-123	EnterWorktree는 cwd만 바꾸지 않고 session context cache를 재기준화한다	code

STUDY SYNTHESIS

Recovery 설계 검증

읽은 내용을 요약하는 대신, 새로운 변경이나 장애 상황에서도 같은 보장을 찾아낼 수 있는지 확인한다.

검증 질문	JSONL chain과 DAG가 동시에 존재하는 이유를 설명한다.
검증 질문	committed-unobserved Edit를 재실행하지 않고 수습한다.
검증 질문	transcript·compaction·session memory·CLAUDE.md의 authority와 cutoff를 비교한다.
검증 질문	parent/task/child truth를 비교한다.
검증 질문	compaction 뒤 다시 읽어야 할 resource를 선정한다.

이 권의 완료 기준

resume와 장기 기억을 “파일 열기”나 “요약 불러오기”가 아니라 여러 evidence projection의 cutoff와 effect truth를 조정하는 reconciliation로 설명한다.

답에는 반드시 현재 snapshot의 코드 근거, 그 근거에서 도출한 해석, 아직 확인할 수 없는 부분을 구분해 적는다.